



Sistemi Distribuiti

Laurea in Informatica

Marco Savi

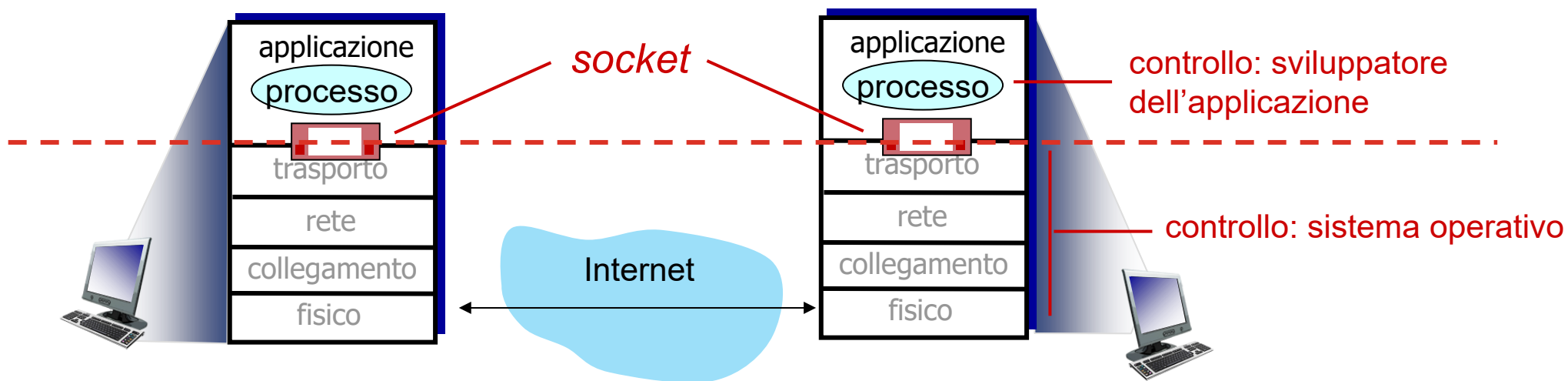
marco.savi@unimib.it



Stream-oriented communication: Le socket

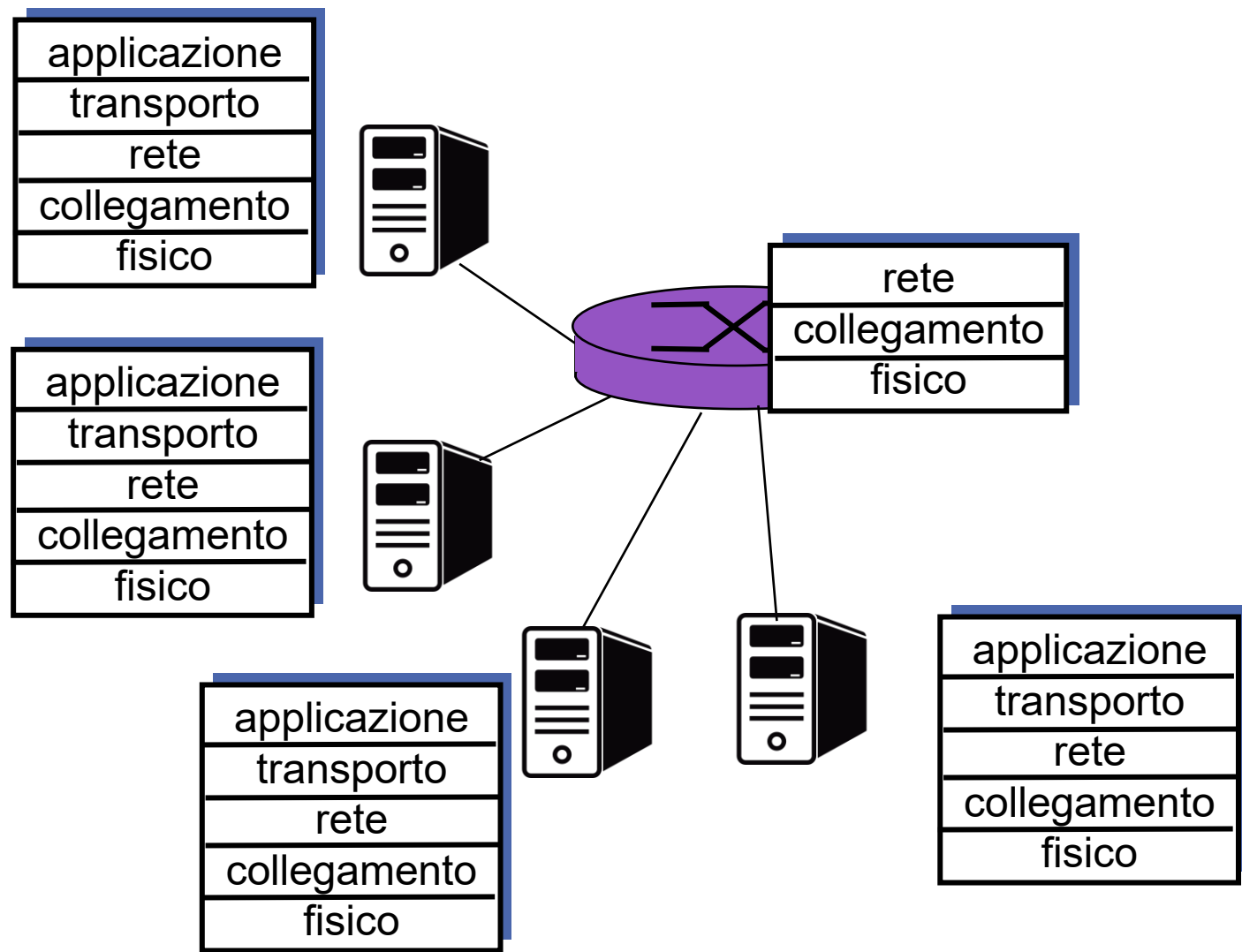
Programmazione delle socket

- Cos'è una socket?
 - Varco tra un processo applicativo e lo strato di trasporto end-to-end



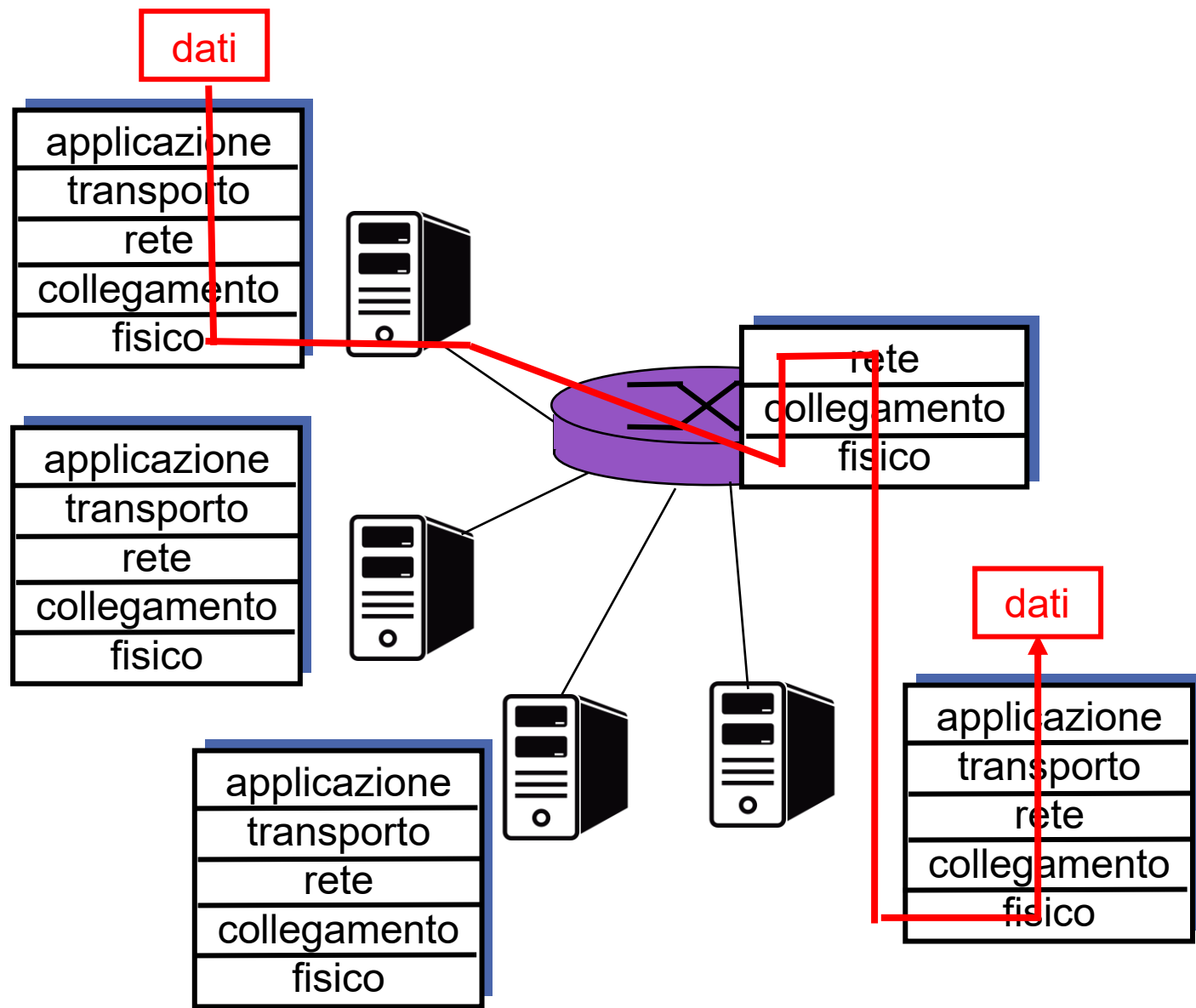
- Obiettivi principali di questa parte di corso:
 - Capire come le socket possono essere controllate dallo sviluppatore
 - Capire come creare applicazioni in grado di comunicare per mezzo delle socket

Architettura a strati



- I **sistemi periferici** (end host) interpretano fino al livello di applicazione
- I **commutatori di pacchetto** interpretano fino al
 - Livello di collegamento (switch)
 - Livello di rete (router)
- **Obiettivo:** trasferire dati tra processi applicativi eseguiti da elaboratori differenti
 - Il **livello di trasporto** offre questo servizio

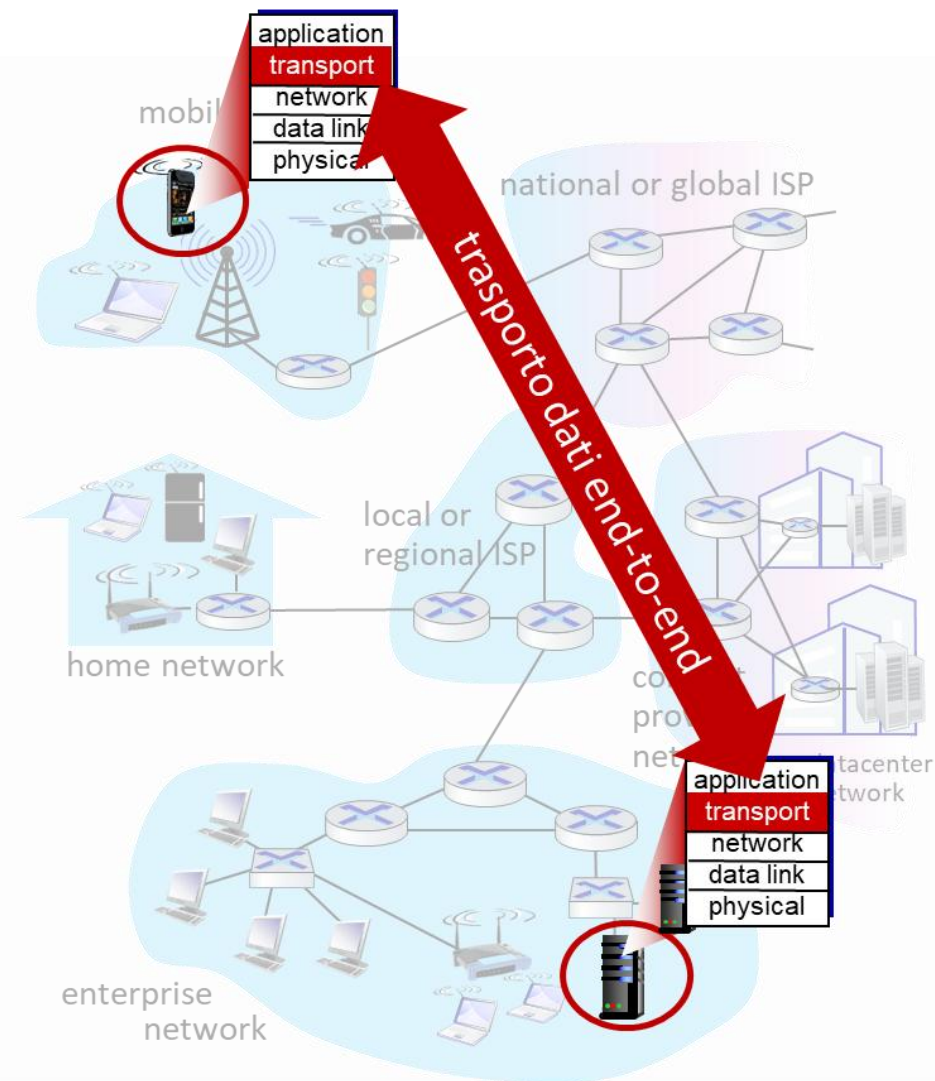
Architettura a strati



- I **sistemi periferici** (end host) interpretano fino al livello di applicazione
- I **commutatori di pacchetto** interpretano fino al
 - Livello di collegamento (switch)
 - Livello di rete (router)
- **Obiettivo:** trasferire dati tra processi applicativi eseguiti da elaboratori differenti
 - Il **livello di trasporto** offre questo servizio

Il livello di trasporto

- Fornisce una **comunicazione logica** tra **processi** applicativi eseguiti da sistemi periferici differenti
- Azioni eseguite dai protocolli di trasporto
 - Mittente: pacchettizza i dati generati dall'applicazione in **segmenti** e li passa al livello di rete
 - Ricevitore: riceve i segmenti e passa i dati estratti al livello di applicazione
- Due protocolli/servizi di trasporto fondamentali
 - TCP e UDP



I servizi di trasporto di Internet: TCP e UDP

Servizio TCP:

- **Orientato alla connessione:**
il client invia al server una richiesta di connessione (three-way handshake)
- **Trasporto dati affidabile:** tutti i dati sono recapitati (in ordine) al processo destinatario
- **Controllo di flusso:** il mittente rallenta l'invio dei dati per non sommergere il ricevente
- **Controllo della congestione:**
il mittente rallenta l'invio dei dati quando la rete è sovraccarica
- **Non offre** garanzie di **banda e ritardo**

Servizio UDP:

- **Trasporto dati non affidabile:** non si ha garanzia di consegna
- **Non è** orientato alla connessione
- **Non effettua** controllo di flusso, controllo di congestione
- **Non offre** garanzie di banda e ritardo

UDP e TCP in atto (policy)

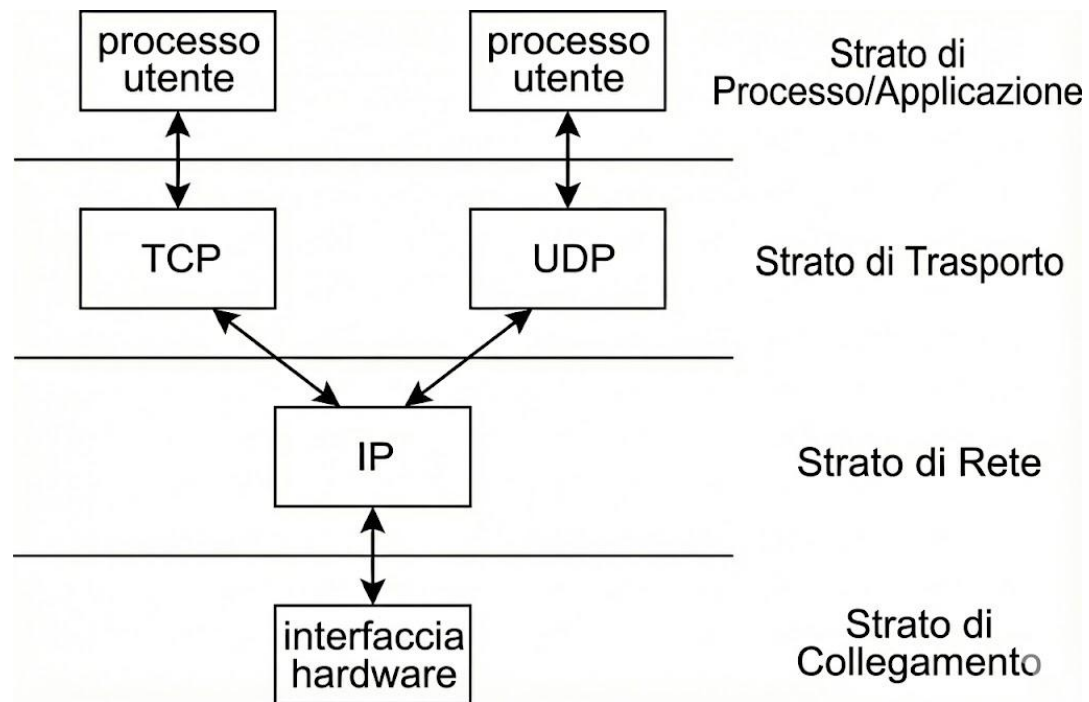
Servizio UDP:

- Scompone il **flusso di byte** in arrivo dal processo applicativo in **segmenti**
- Li passa, uno per volta, ai servizi di rete per la consegna al destinatario

Servizio TCP:

- Scompone il **flusso di byte** e passa i **segmenti** ai servizi di rete come UDP
- I **byte** vengono **numerati** progressivamente per garantire:
 - Il loro riordinamento una volta arrivati al destinatario
 - Il controllo dei duplicati → Scarto dei byte con stesso numero di sequenza
 - Il controllo delle perdite → Rinvio dei segmenti relativi a byte mancanti
- **Nota:** per progettare e realizzare **sistemi distribuiti**
 - Non è necessario conoscere il funzionamento dettagliato dei servizi TCP e UDP
 - E' necessario sapere «solo» cosa offrono all'applicazione

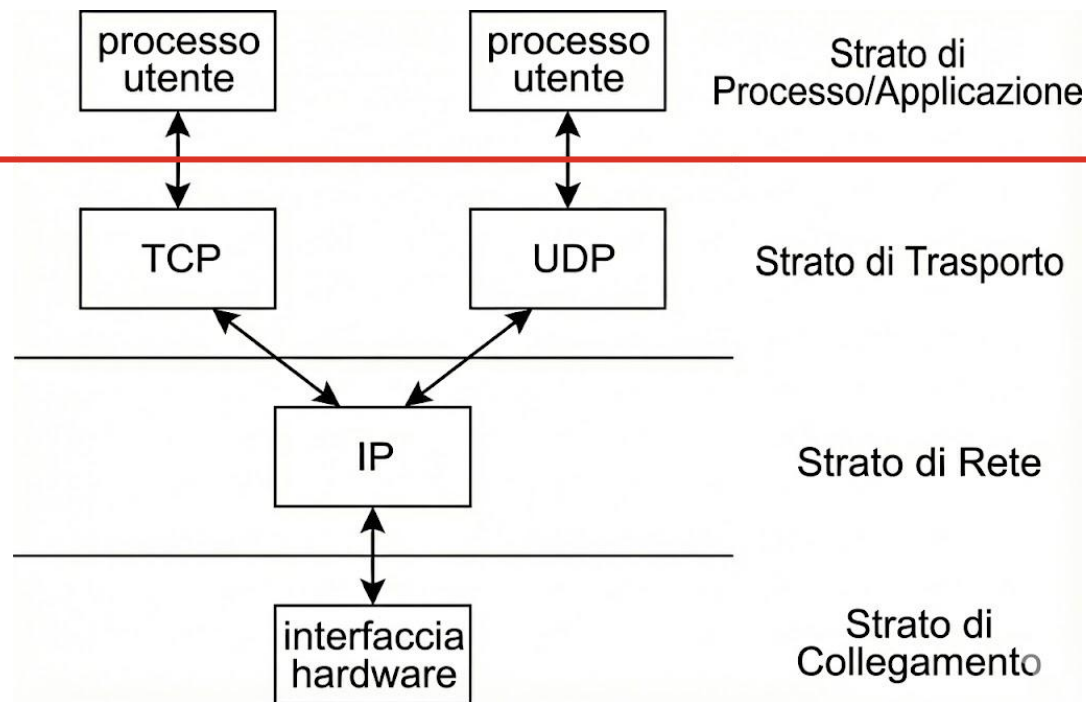
Reti e sistemi distribuiti



Reti e sistemi distribuiti

“Regno” dei **sistemi distribuiti**

“Regno” delle **reti**



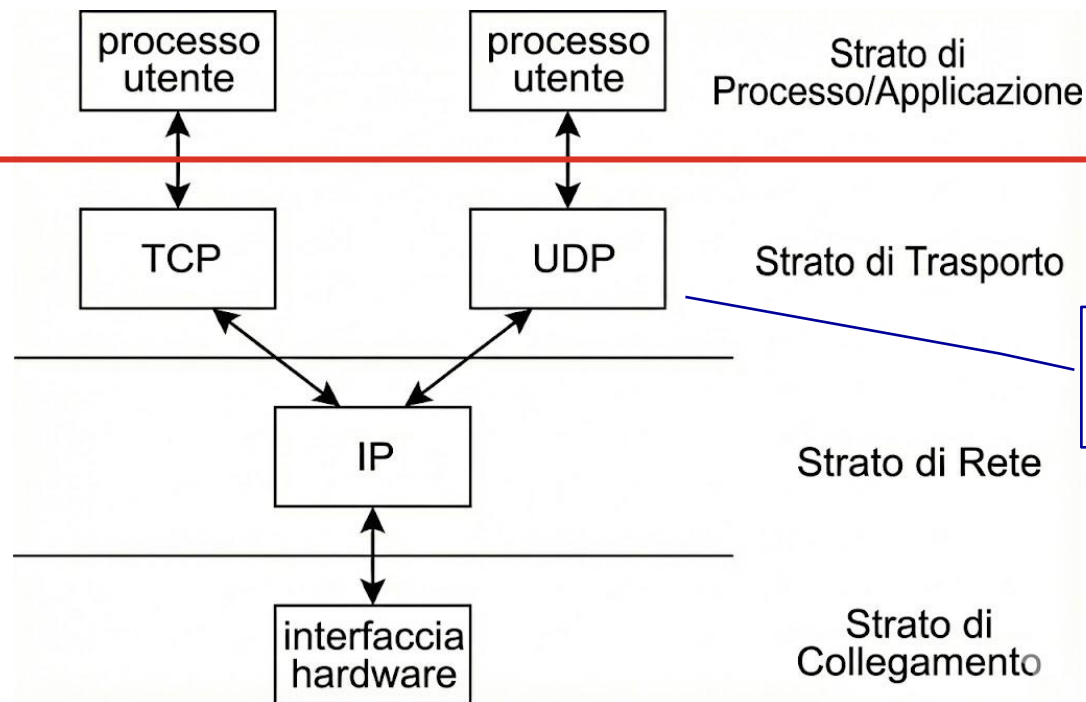
Spazio utente

Spazio kernel

Reti e sistemi distribuiti

“Regno” dei **sistemi distribuiti**

“Regno” delle **reti**

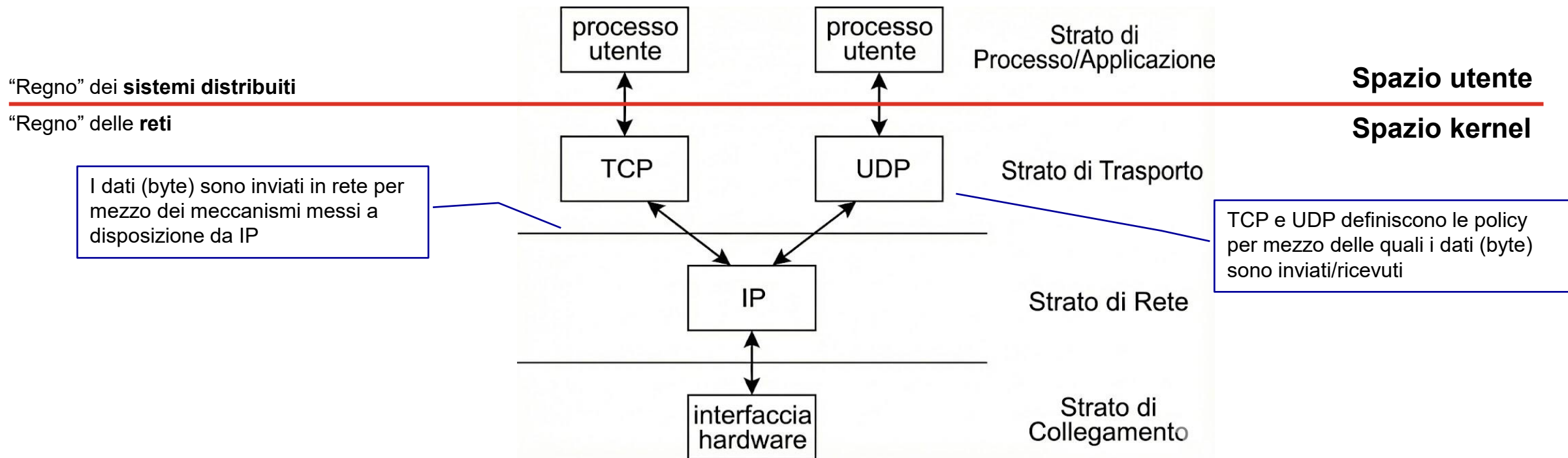


Spazio utente

Spazio kernel

TCP e UDP definiscono le policy per mezzo delle quali i dati (byte) sono inviati/ricevuti

Reti e sistemi distribuiti



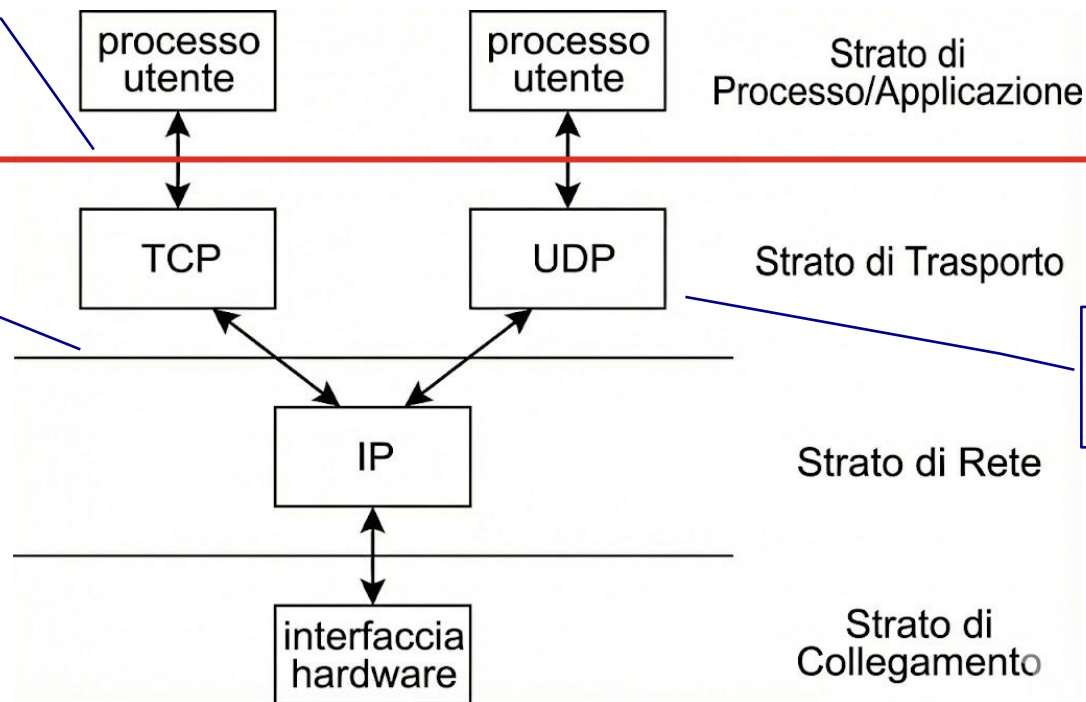
Reti e sistemi distribuiti

I processi utente possono inviare/ricevere flussi di byte per mezzo di API specifiche

“Regno” dei **sistemi distribuiti**

“Regno” delle **reti**

I dati (byte) sono inviati in rete per mezzo dei meccanismi messi a disposizione da IP



Strato di
Processo/Applicazione

Spazio utente

Spazio kernel

Strato di Trasporto

TCP e UDP definiscono le policy per mezzo delle quali i dati (byte) sono inviati/ricevuti

Strato di Rete

Strato di
Collegamento

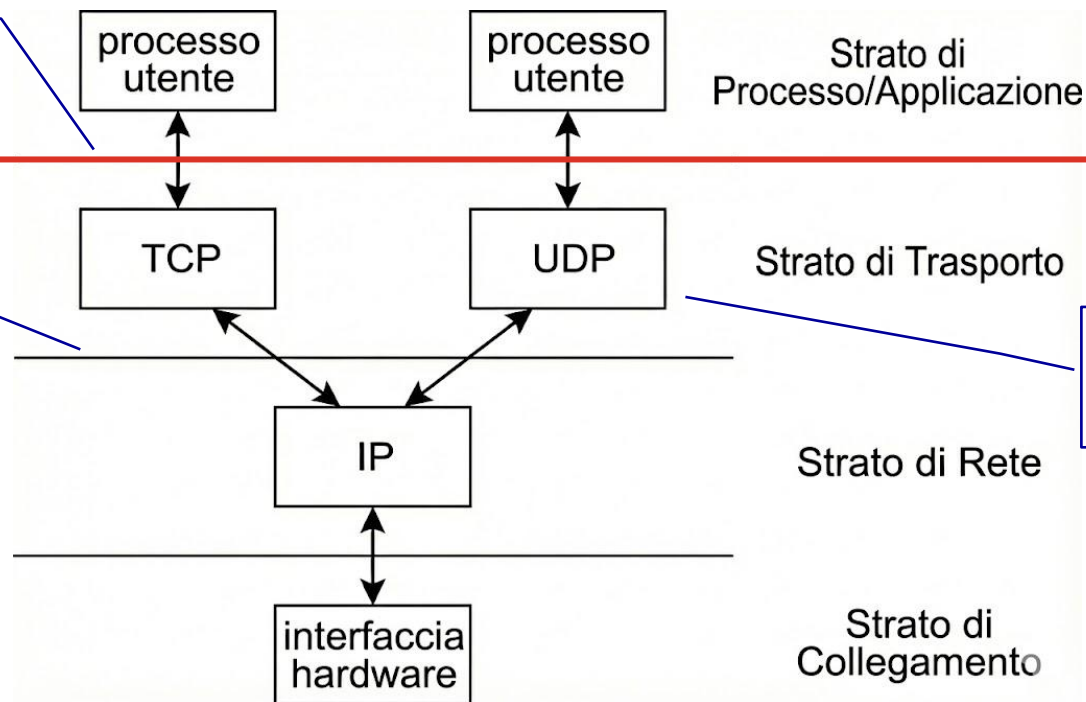
Reti e sistemi distribuiti

I processi utente possono inviare/ricevere flussi di byte per mezzo di API specifiche

“Regno” dei **sistemi distribuiti**

“Regno” delle **reti**

I dati (byte) sono inviati in rete per mezzo dei meccanismi messi a disposizione da IP



TCP e UDP definiscono le policy per mezzo delle quali i dati (byte) sono inviati/ricevuti

■ Cos'è una **Application Programming Interface (API)**?

- Insieme di regole e messaggi che permettono a due software diversi di comunicare e scambiarsi informazioni in modo standardizzato

■ Nel caso specifico: implementate attraverso **chiamate di sistema**

- Interazione tra spazio utente e spazio kernel

Socket in TCP e UDP: funzionamento di base e ruoli

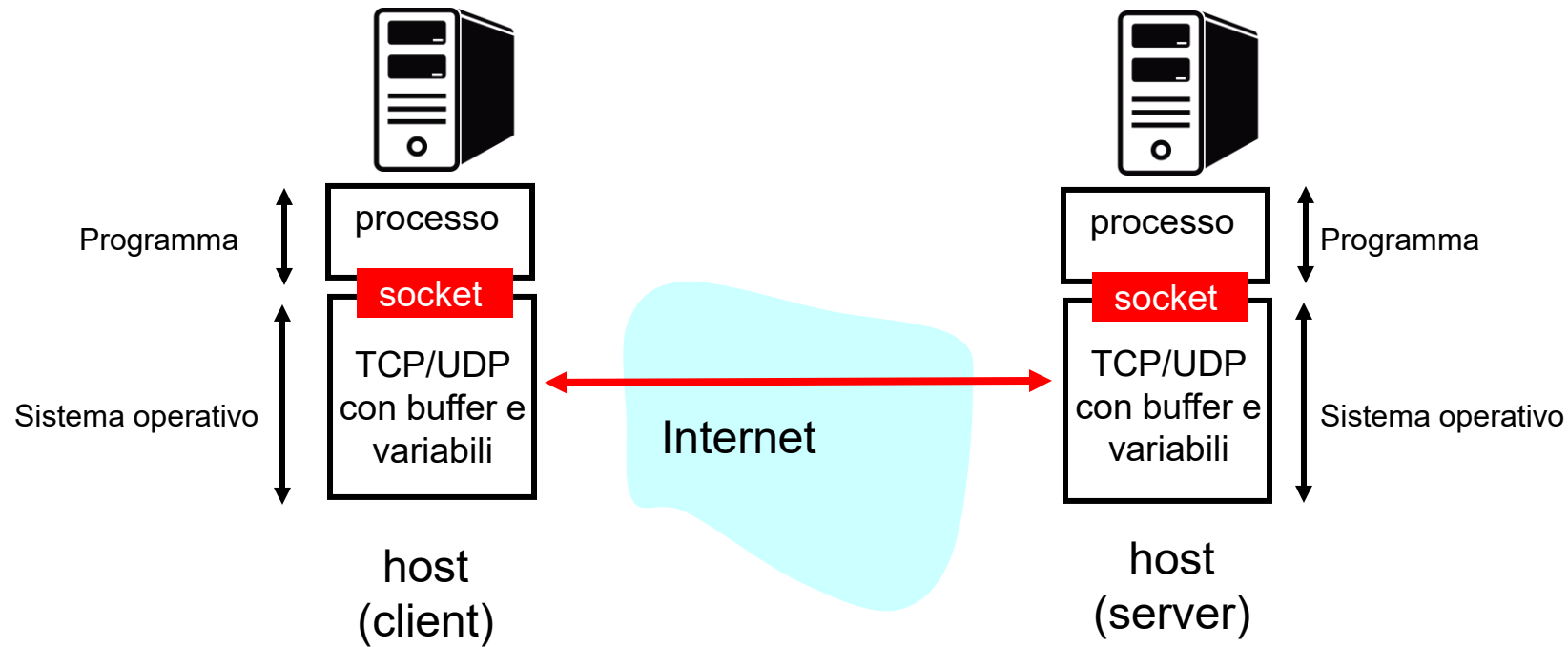
TCP

- Alloca variabili e buffer per realizzare il trasferimento bidirezionale di **flussi di byte** tra processi
- Prevede ruoli client/server durante la connessione
- Non prevede necessariamente ruoli client/server per la comunicazione/conversazione (modello peer-to-peer)

UDP

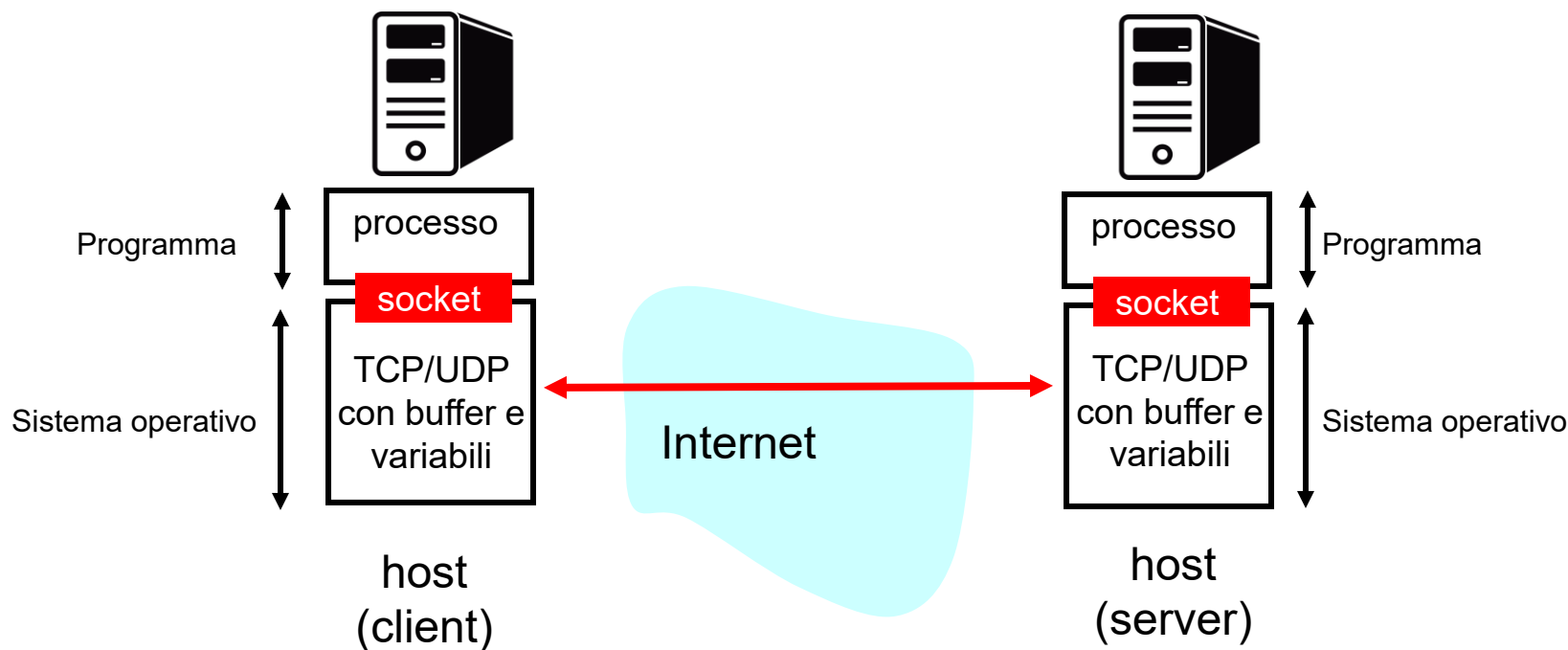
- Alloca variabili (in misura molto minore) e buffer per realizzare il trasferimento di **flussi di byte** tra processi
- Può prevedere ruoli client/server durante la comunicazione/conversazione (ma in maniera fluida e strettamente dipendente dalla logica dell'applicazione)
- D'ora in poi: consideriamo un **modello client-server** anche in fase di conversazione

Socket: funzionamento di base



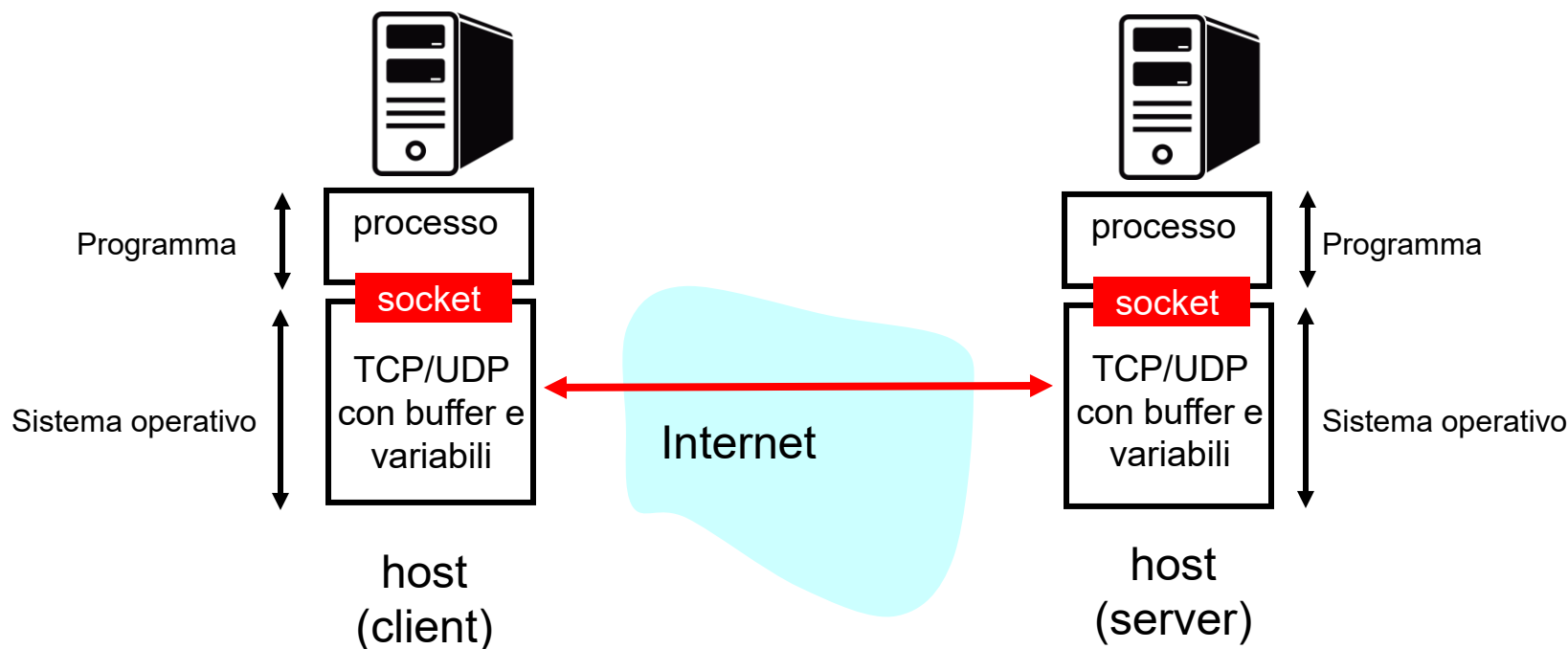
- L'allocazione di **buffer** e **variabili** per TCP e UDP avviene a livello di **sistema operativo** (implementazione dello stack TCP/IP)
 - Buffer fondamentali: buffer di invio e buffer di ricezione

Socket: funzionamento di base



- I **programmi** vengono eseguiti dai processi
 - Un programma è una sequenza di istruzioni eseguibili dalla «macchina»

Socket: funzionamento di base

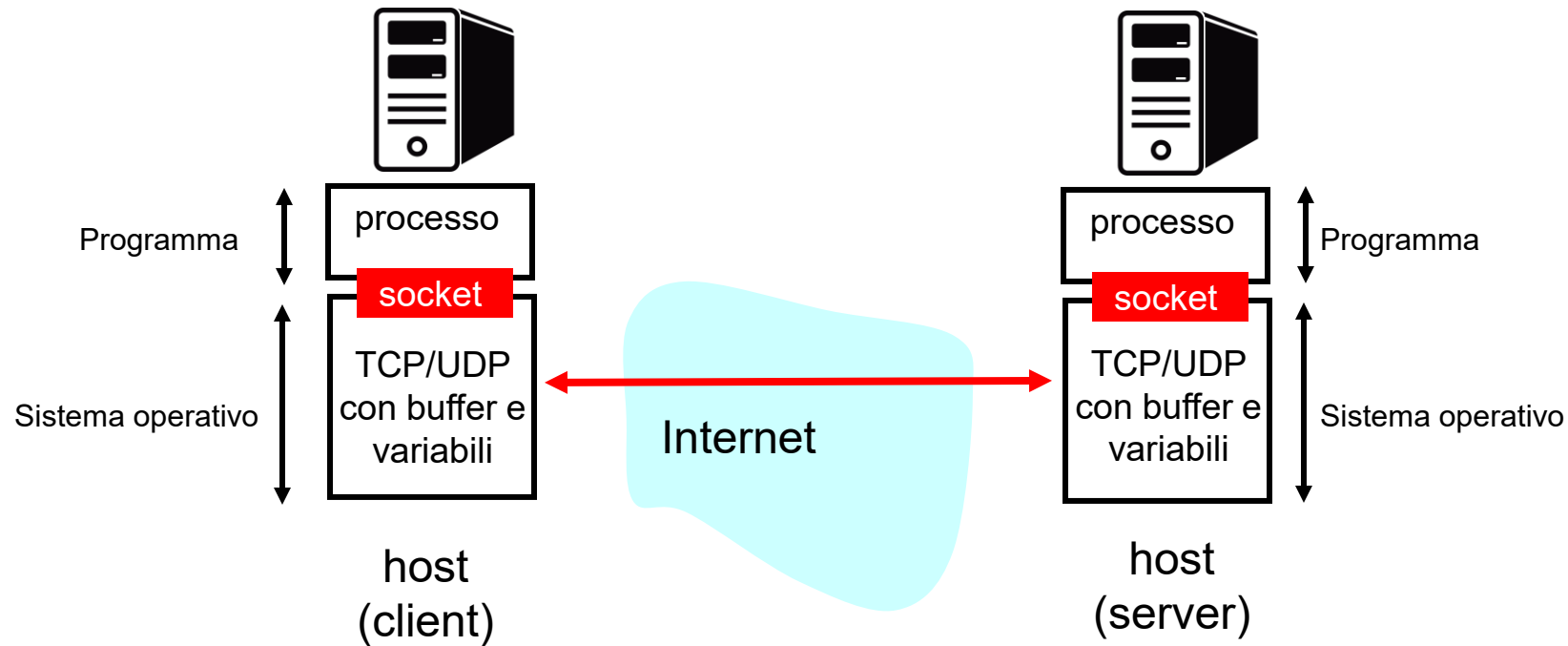


- I **processi** sono entità gestite dal sistema operativo (programmi in esecuzione)
- Un processo fa affidamento su:
 - Area di memoria RAM per effettuare le operazioni e memorizzare i dati
 - Registro che ricorda la prossima istruzione da eseguire (per gestire context switch)
 - Canali di comunicazione

Breve excursus: i canali di Input/Output (I/O)

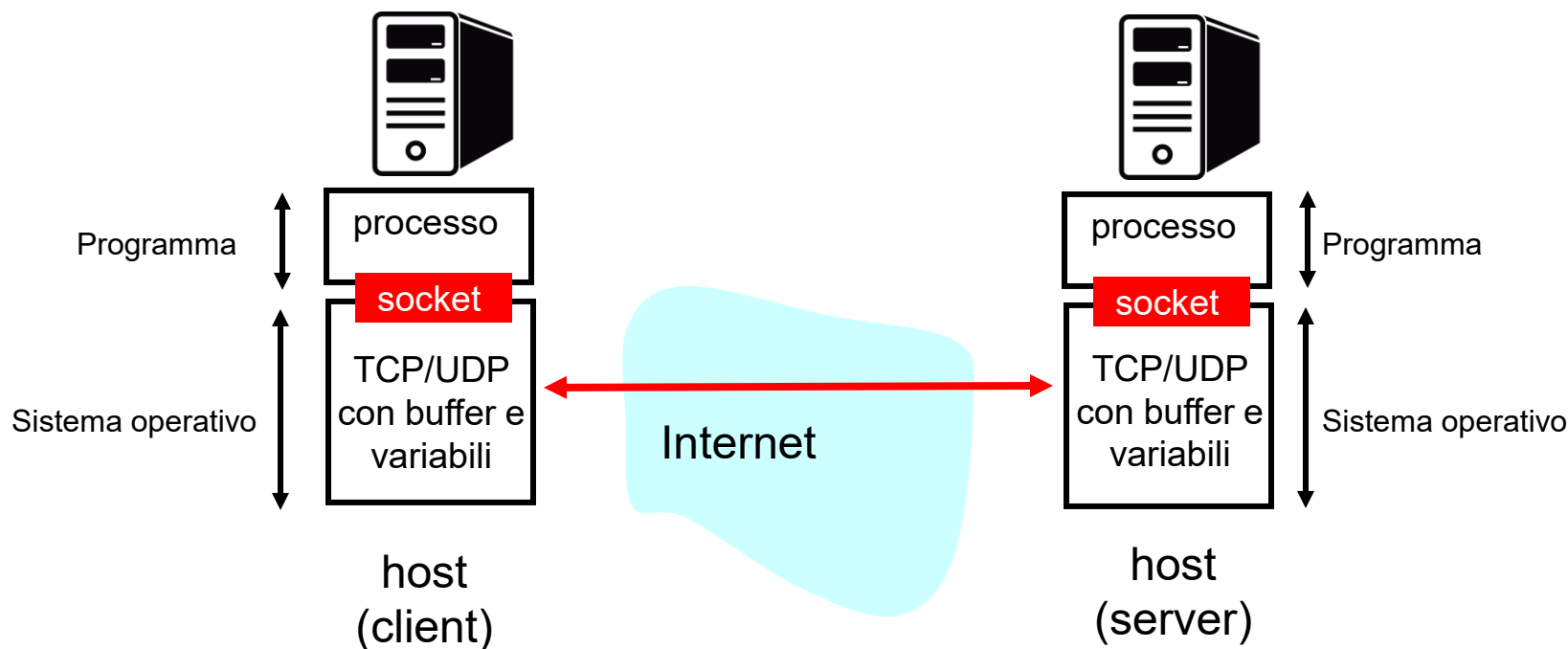
- Ogni processo comunica attraverso **canali di Input/Output (I/O)**
 - Un **canale** è un'**astrazione** che rappresenta il modo in cui vengono gestiti i flussi di dati in ingresso e in uscita rispetto al processo
- Il sistema operativo si serve dei canali di I/O per fungere da intermediario tra il «mondo esterno» (es. tastiera, schermo, rete, disco) e il processo
 - Ingresso: Mondo esterno → Canale → Processo
 - Esempi: tastiera, rete, disco
 - Uscita: Processo → Canale → Mondo esterno
 - Esempi: schermo, rete, disco
- Ogni canale è univocamente identificato a livello di sistema operativo da un **numero**
 - «File Descriptor» in UNIX
 - «Handle» in Windows

Socket: funzionamento di base



- La **socket** è un particolare canale di I/O per la comunicazione tra processi che non condividono memoria
 - Endpoint su cui un processo può scrivere dati da inviare e può leggere dati ricevuti
 - Per il sistema operativo non è altro che un «file»!
 - Per interagire con le socket (con TCP) si usano le stesse chiamate di sistema che si adottate per scrivere/leggere dati su/da un file (lo vediamo tra poco)

Socket: funzionamento di base



- Per la comunicazione con l'esterno, una **socket** è associata a un numero di **porta**
- Per potersi connettere o inviare dati ad un processo A, un processo B deve conoscere l'**indirizzo IP dell'host** che esegue A e la **porta** della socket usata da A

Definizione di client e server: aspetti critici

- Gestione del **ciclo di vita** di client e server
 - Chi attiva/termina il client e il server? (Es. attivazione manuale o gestita da un middleware?)
- **Identificazione e accesso al server**
 - Quali informazioni deve conoscere il client per identificare e accedere al server?
- **Comunicazione** tra client e server
 - Quali chiamate di sistema sono disponibili e quali modalità devono essere adottate per la comunicazione?
- **Ripartizione dei compiti** tra client e server
 - In che modo viene distribuita la logica dell'applicazione tra client e server?
 - Dipende dal tipo di applicazione (es. una banca per questioni di controllo vorrebbe gestire tutto lato server)
 - Influenza la prestazioni in relazione al carico (numero di client)

Definizione di client e server: aspetti critici

- Come fa il client a conoscere l'indirizzo del server?
- Alternative:
 - Inserire nel codice del client l'indirizzo del server espresso come costante (es. client di un servizio bancario)
 - Chiedere all'utente l'indirizzo (es. web browser)
 - Utilizzare un name server o un repository da cui il client può acquisire le informazioni necessarie (es. Domain Name System (DNS) per tradurre hostname in indirizzi IP)
 - Adottare un protocollo e una modalità di comunicazione specifica per l'individuazione del server (es. broadcast per DHCP)

Problematiche fondamentali dei sistemi distribuiti e TCP/IP

- In che modo le problematiche fondamentali dei sistemi distribuiti sono affrontate quando lo stack TCP/IP è adottato per la comunicazione?
1. Identificazione della controparte (**naming**)
 - Identificazione di basso livello: servono l'indirizzo IP e il numero di porta per identificare il processo
 2. Accedere alla controparte (**access point**)
 - Accesso al processo per mezzo della socket, identificato dalla coppia {indirizzo IP, porta}
 3. Comunicazione 1 (**protocol**)
 - I processi si scambiano flussi di bytes
 4. Comunicazione 2 (**syntax and semantics**)
 - Vengono adottati protocolli applicativi con una sintassi e semantica predefinita (es. HTTP)

Problematiche fondamentali dei sistemi distribuiti e TCP/IP

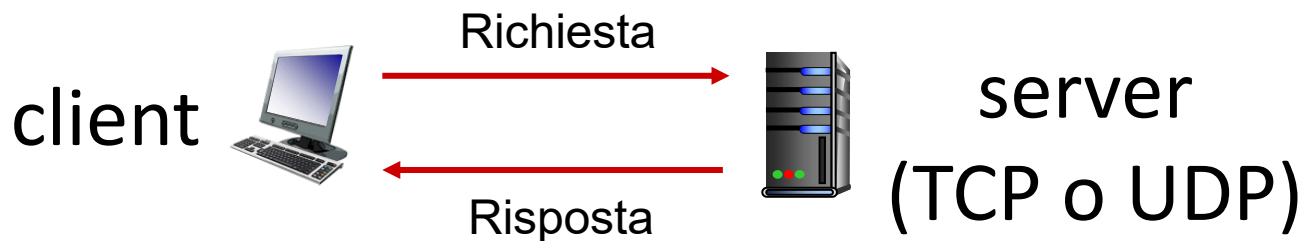
- Adottando lo stack TCP/IP per l'implementazione di un sistema distribuito, qual è il **livello di trasparenza** per il programmatore?

Problematiche fondamentali dei sistemi distribuiti e TCP/IP

- Adottando lo stack TCP/IP per l'implementazione di un sistema distribuito, qual è il **livello di trasparenza** per il programmatore?
 - **Molto basso**, in quanto il programmatore deve:
 - Conoscere gli indirizzi di rete (indirizzo IP e porta di destinazione)
 - Effettuare il parsing dei byte per ottenere il contenuto trasportato dai segmenti

Comunicazione via socket

- **Problema:** scrivere un'applicazione che adotti un **modello client-server** e comunichi sfruttando i servizi messi a disposizione del livello di trasporto
- **Soluzione:** utilizzo di un'interfaccia (API) standardizzata
 - **Berkeley (BSD) Sockets**, introdotte negli anni 70
 - Diventate successivamente (con modifiche minimali) standard POSIX
 - Permettono di effettuare svariate **chiamate di sistema** per accedere ai servizi TCP e UDP
- Di seguito vediamo come le Berkeley Sockets vengono utilizzate per far interagire un **client** con
 - Un **server** UDP
 - Un **server** TCP



Interazione client/server: UDP (schema)



server (*serverIP*)



client (*clientIP*)

crea socket su numero di porta x (**server socket**)

crea socket (**client socket**)

ricevi la richiesta da **server socket**

crea richiesta con indirizzo di destinazione *serverIP* e numero di porta x; invia la richiesta via **client socket**

invia la risposta via **server socket** con indirizzo di destinazione *clientIP* e numero di porta del client

ricevi la risposta da **client socket**

chiudi **client socket**

richiesta

risposta

Interazione client/server: UDP (schema)



server (*serverIP*)



client (*clientIP*)

crea socket su numero di porta x (**server socket**)

crea socket (**client socket**)

ricevi la richiesta da **server socket**

crea richiesta con indirizzo di destinazione *serverIP* e numero di porta x; invia la richiesta via **client socket**

invia la risposta via **server socket** con indirizzo di destinazione *clientIP* e numero di porta del client

ricevi la risposta da **client socket**

chiudi **client socket**

richiesta

risposta

■ Qualcosa di strano?

Interazione client/server: UDP (schema)



server (*serverIP*)



client (*clientIP*)

crea socket su numero di porta x (**server socket**)

crea socket (**client socket**)

ricevi la richiesta da **server socket**

crea richiesta con indirizzo di destinazione *serverIP* e numero di porta x; invia la richiesta via **client socket**

invia la risposta via **server socket**
con indirizzo di destinazione
clientIP e numero di porta del
client

richiesta

risposta

ricevi la risposta da **client socket**

chiudi **client socket**

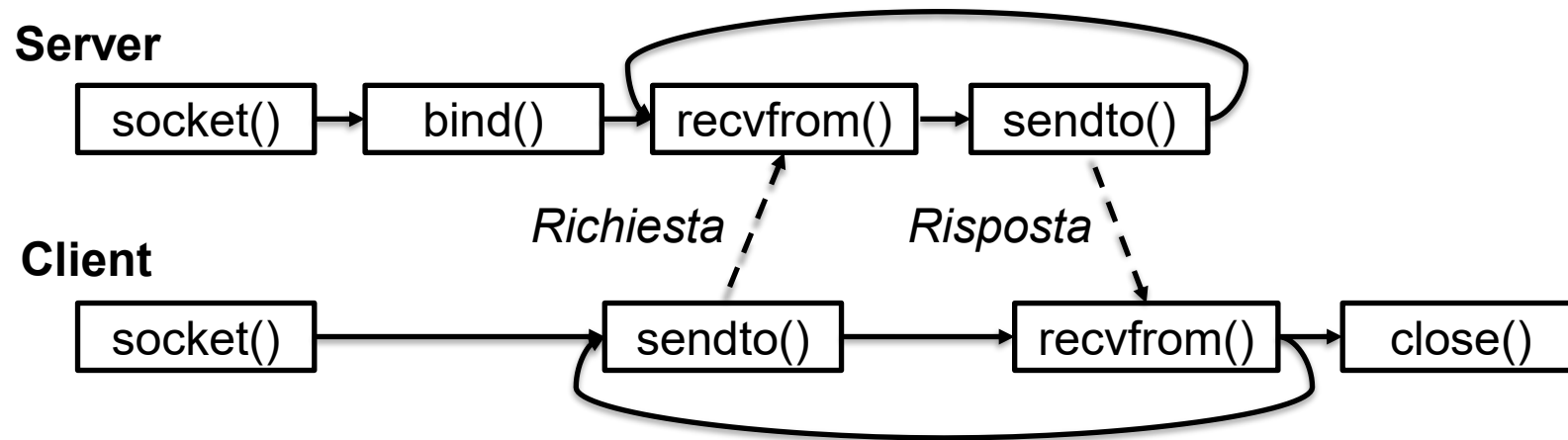
■ Qualcosa di strano?

- Il numero di porta del client viene scelto dal sistema operativo!

Breve excursus: i numeri di porta

- I campi **porta sorgente** e **porta destinazione** nelle intestazioni TCP e UDP sono da 16 byte
 - Numeri di porta da 0 a 65535
- Numeri di porta suddivisi nei seguenti **range**:
 - **Porte ben note** (well-known ports): 0-1023
 - Assegnate a servizi e protocolli comuni (es. HTTP: 80, DNS: 53, DHCP server: 68, ...)
 - La IANA (Internet Assigned Numbers Authority) decide come assegnare le porte
 - **Porte registrate** (registered ports): 1024-49151
 - Utilizzate da aziende o applicazioni specifiche per registrare servizi
 - Si garantisce non ci siano conflitti
 - Anche in questo caso assegnate dalla IANA
 - **Porte dinamiche o effimere** (dynamic/ephemeral ports): 49152-65535
 - Liberamente utilizzabili dai sistemi operativi per le per applicazioni client che richiedono una porta temporanea
 - Caso della slide precedente
- **Importante:** i range TCP e UDP sono indipendenti: la porta 80 TCP è diversa dalla porta 80 UDP (che comunque non viene tipicamente utilizzata)

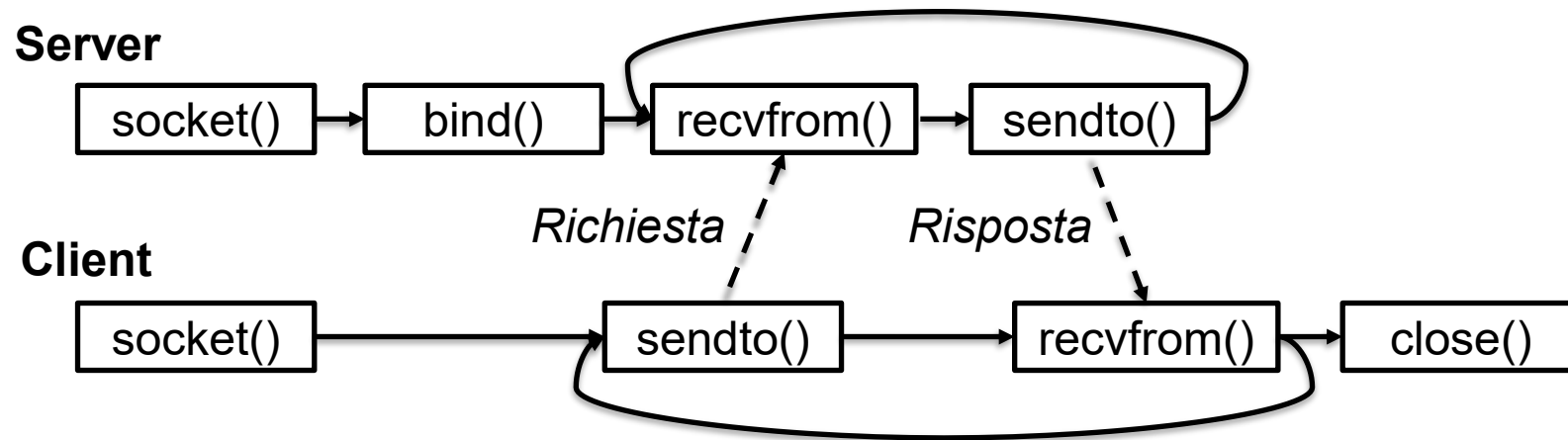
Interazione client/server: UDP (pattern di comunicazione ed API)



■ Il server UDP:

1. Crea una socket UDP per mezzo della chiamata di sistema **socket()**
 - Il sistema operativo alloca le risorse necessarie (es. buffer di invio e buffer di ricezione)
2. Associa la socket all'indirizzo IP e alla porta del server per mezzo di **bind()**
 - Il sistema operativo esegue vari controlli (es. che la porta non sia già utilizzata)
3. Si mette in attesa della ricezione di una richiesta per mezzo di **recvfrom()**
4. Una volta ricevuta, la elabora e invia la risposta per mezzo di **sendto()**
5. Ritorna al punto 3

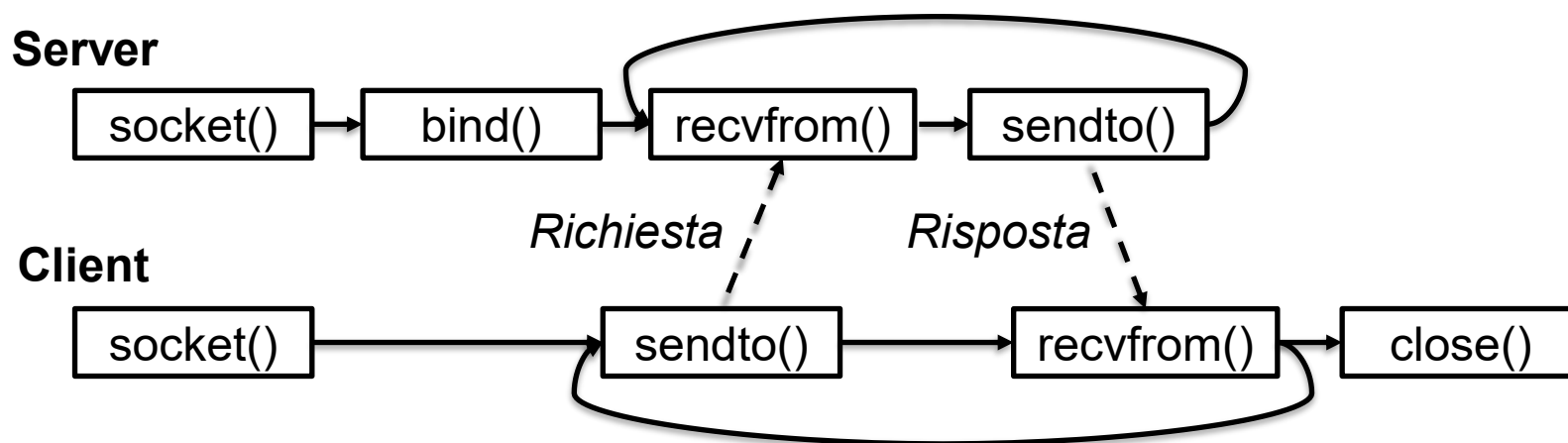
Interazione client/server: UDP (pattern di comunicazione ed API)



■ Il client UDP:

1. Crea una socket UDP per mezzo della chiamata di sistema **socket()**
 - Il sistema operativo alloca le risorse necessarie (es. buffer di invio e buffer di ricezione), tra cui la porta scelta per la comunicazione
2. Invia la richiesta per mezzo di **sendto()**
3. Si mette in attesa della ricezione della risposta per mezzo di **recvfrom()**
4. Ritorna al punto 2
5. Chiude la socket per mezzo di **close()**
 - Il sistema operativo dealloca le risorse non più necessarie

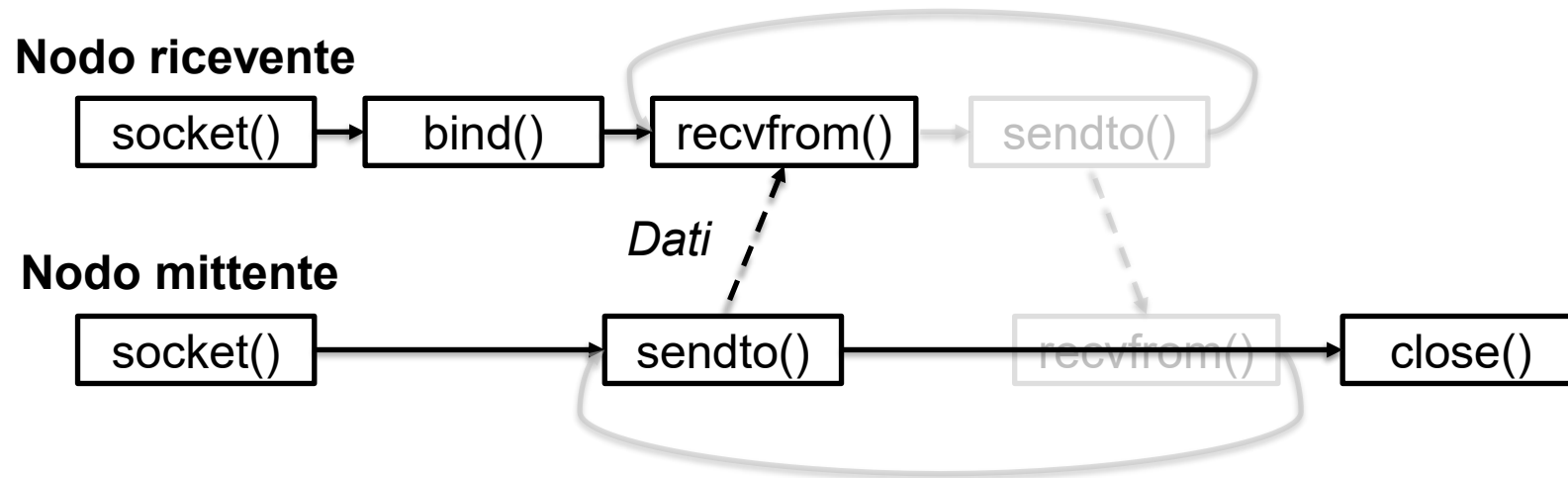
Interazione client/server: UDP (pattern di comunicazione ed API)



- **Importante!** La chiamata di sistema **recvfrom()** è **bloccante** (o sospensiva)
 - L'esecuzione del programma si ferma e il controllo è ceduto al sistema operativo finché l'operazione non è completata (o si verifica un evento specifico)
 - **recvfrom()** blocca l'esecuzione finché non ci sono dati da prelevare nel buffer (o evento di errore)
- Le altre chiamate di sistema **non** sono bloccanti
 - Possibile eccezione: **sendto()** → Può bloccare se il buffer di invio è pieno
- Per l'interazione client/server UDP in Java **non** vediamo esempi a lezione
 - Lezione di **laboratorio** a riguardo

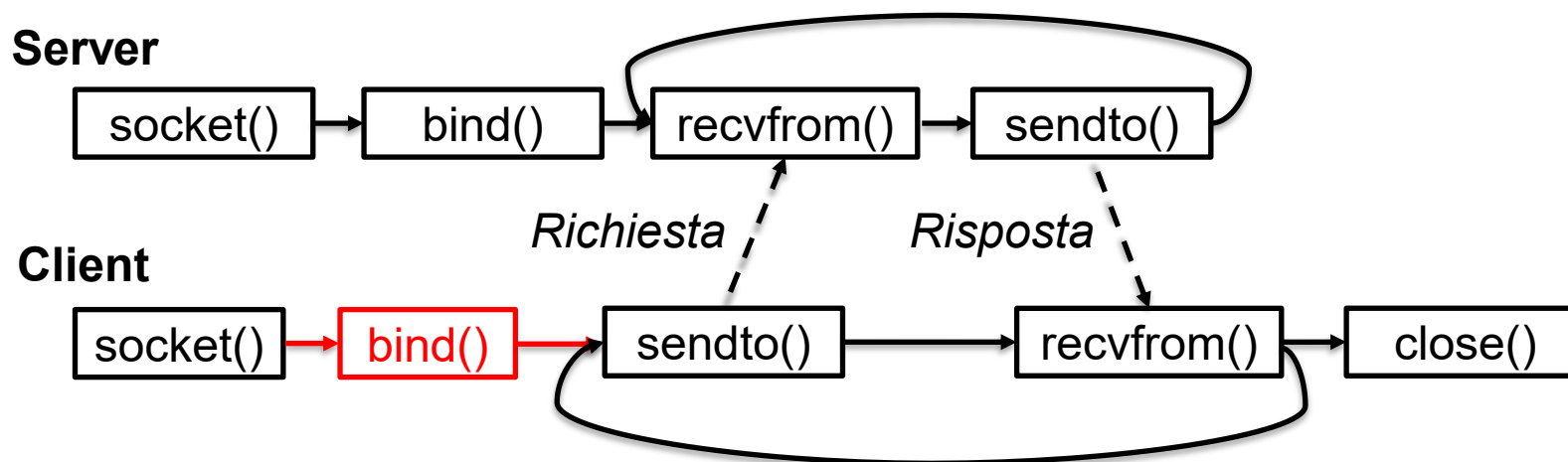
Interazione client/server: UDP (pattern di comunicazione ed API)

- **Domanda:** è sempre necessario seguire il pattern di comunicazione introdotto precedentemente con UDP?
 - No, se non vogliamo seguire un modello client-server o in **casi specifici**
- **Esempio 1:**
 - Nodo che vuole inviare dati a un altro nodo senza necessità di ottenere una risposta (es. misurazione di un sensore inviata verso un gateway)



Interazione client/server: UDP (pattern di comunicazione ed API)

- **Domanda:** è sempre necessario seguire il pattern di comunicazione introdotto precedentemente con UDP?
 - No, se **non** vogliamo seguire un **modello client-server** o in **casi specifici**
- **Esempio 2:**
 - Servizio DHCP (porta nota per client numero 68)



Interazione client/server: TCP (schema)

Legenda:

—→ Connessione
—→ Conversazione



server (*serverIP*)



client (*clientIP*)

crea socket su numero di porta x (**listening socket**, o **socket di ascolto**)

crea socket (**client socket**)

attendi una richiesta di connessione in ingresso, quindi crea una nuova socket (**connection socket**, o **socket di servizio**)

crea richiesta di connessione con indirizzo di destinazione *serverIP* e numero di porta x

setup connessione TCP



ricevi la richiesta da **connection socket**

invia la richiesta via **client socket** (destinazione *serverIP* e porta x)

richiesta

invia la risposta via **connection socket** (destinazione *clientIP* e porta client)

ricevi la risposta da **client socket**

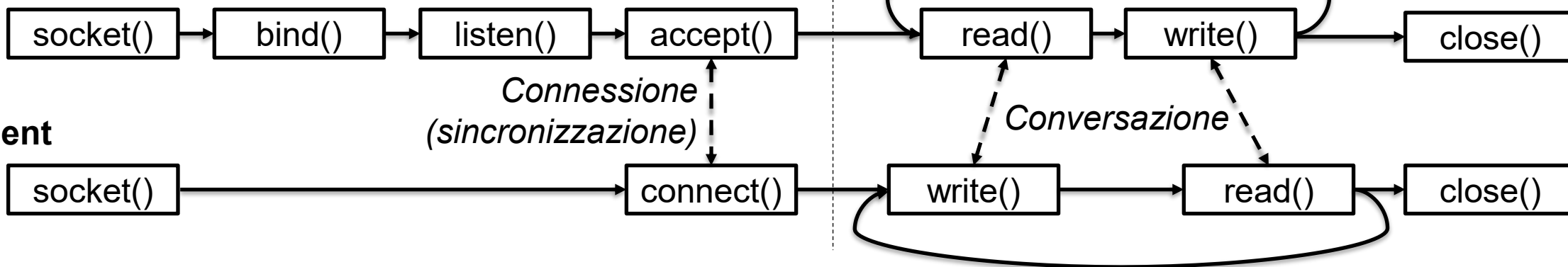
risposta

chiudi **connection socket**

chiudi **client socket**

Interazione client/server: TCP (pattern di comunicazione ed API)

Server

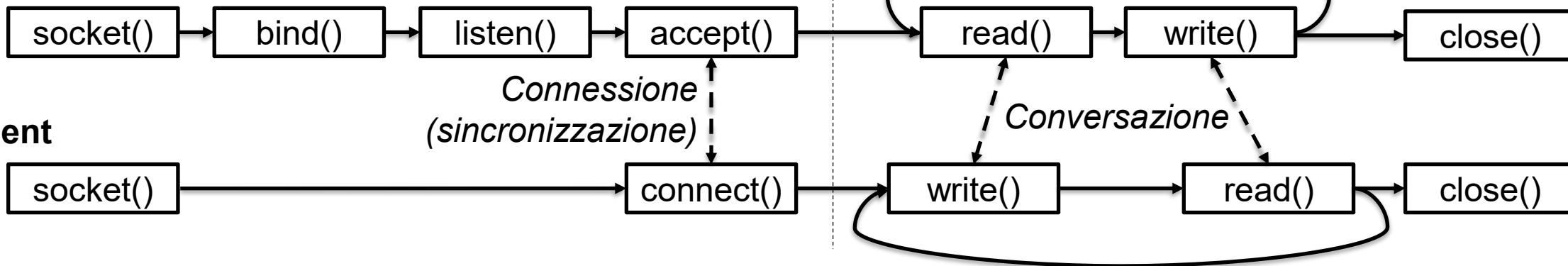


■ Il server TCP (fase di connessione):

1. Crea una **socket di ascolto** per mezzo della chiamata di sistema **socket()**
2. Associa la socket all'indirizzo IP e alla porta del server per mezzo di **bind()**
3. Per mezzo di **listen()** la socket richiede al sistema operativo di allocare i buffer necessari per garantire che possano essere ricevute richieste di connessione
 - Due buffer (spesso chiamati **backlog**):
 - **Incomplete Connection Queue**: Contiene le connessioni per le quali il server ha inviato un SYNACK, ma non ha ancora ricevuto l'ACK finale dal client
 - **Completed Connection Queue**: Contiene le connessioni che hanno completato l'handshake

Interazione client/server: TCP (pattern di comunicazione ed API)

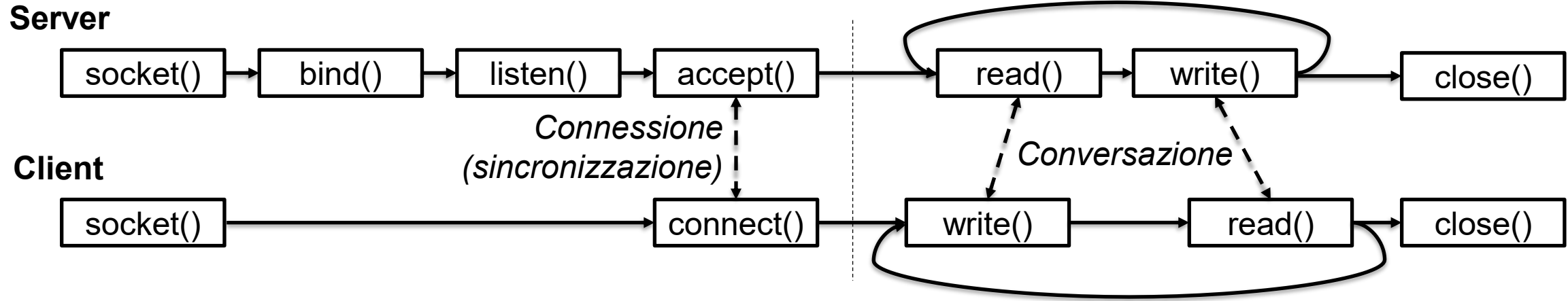
Server



■ Il server TCP (fase di connessione):

4. Si mette in attesa della ricezione di una richiesta di connessione per mezzo di **accept()**
 - Accept “estrae” una connessione accodata nella Completed Connection Queue (per cui è stato quindi finalizzato il three-way handshake)
 - Crea una **socket di servizio** che può essere usata per la conversazione
 - Il sistema operativo alloca le risorse necessarie per tale socket (es. buffer di invio e di ricezione)

Interazione client/server: TCP (pattern di comunicazione ed API)

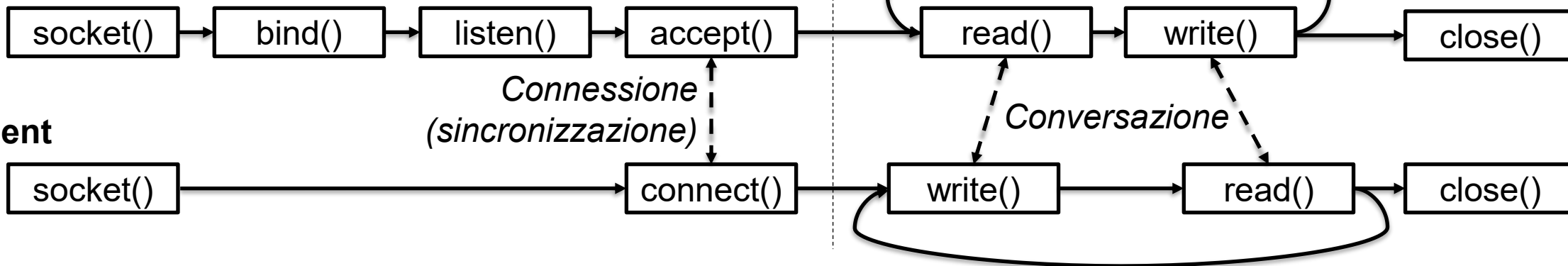


■ Il server TCP (fase di conversazione):

5. Si mette in attesa della ricezione di una richiesta per mezzo di **read()**
6. Una volta ricevuta, la elabora e invia la risposta per mezzo di **write()**
7. Ritorna al punto 5
8. Al termine della conversazione chiude la socket di servizio per mezzo di **close()**
 - Vari casi: chiamata esplicita, messaggio di FIN, timeout, ...
9. Ritorna al punto 4

Interazione client/server: TCP (pattern di comunicazione ed API)

Server

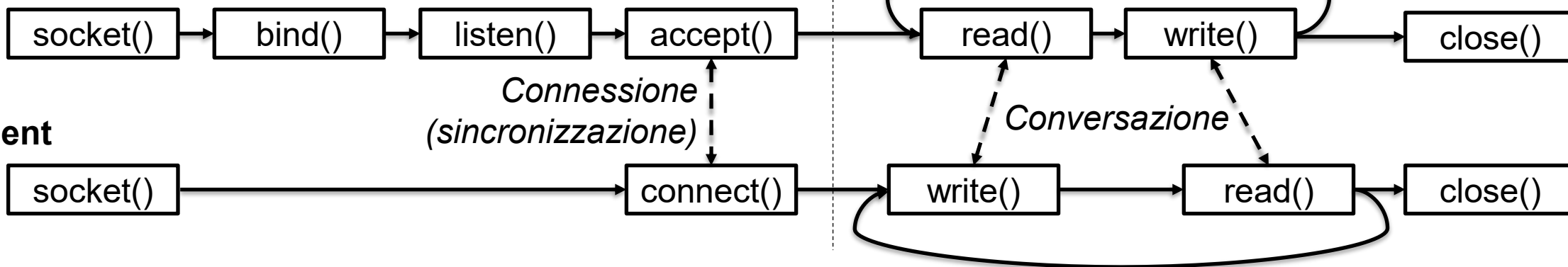


■ Il client TCP (fase di connessione):

1. Crea una socket TCP per mezzo della chiamata di sistema **socket()**
 - Anche in questo caso (come per UDP) la **porta** è scelta dal sistema operativo!
2. Invia una richiesta di connessione per mezzo di **connect()**
 - Vengono specificati l'indirizzo IP e la porta di destinazione, viene avviata dal sistema operativo la procedura di three-way handshake

Interazione client/server: TCP (pattern di comunicazione ed API)

Server

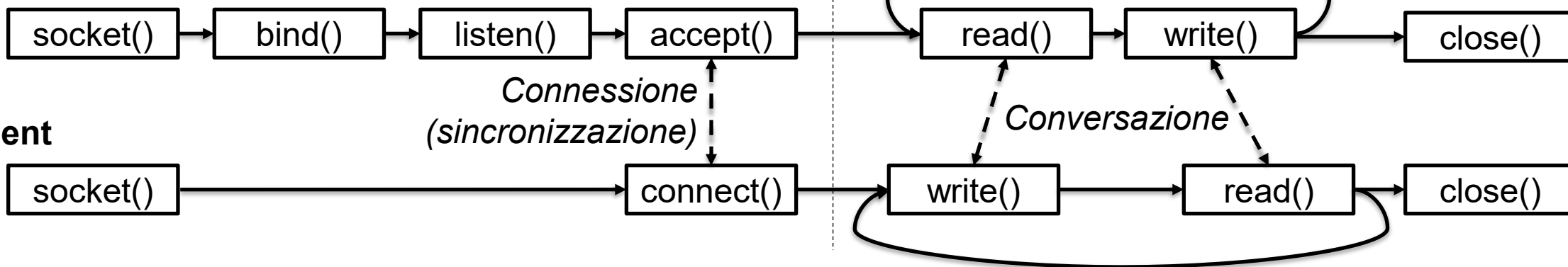


■ Il client TCP (fase di conversazione):

3. Invia una richiesta per mezzo di **write()**
4. Si mette in attesa della ricezione della risposta per mezzo di **read()**
5. Torna al punto 3
6. Al termine della conversazione chiude la socket per mezzo di **close()**
 - Vari casi: chiamata esplicita, messaggio di FIN, timeout, ...

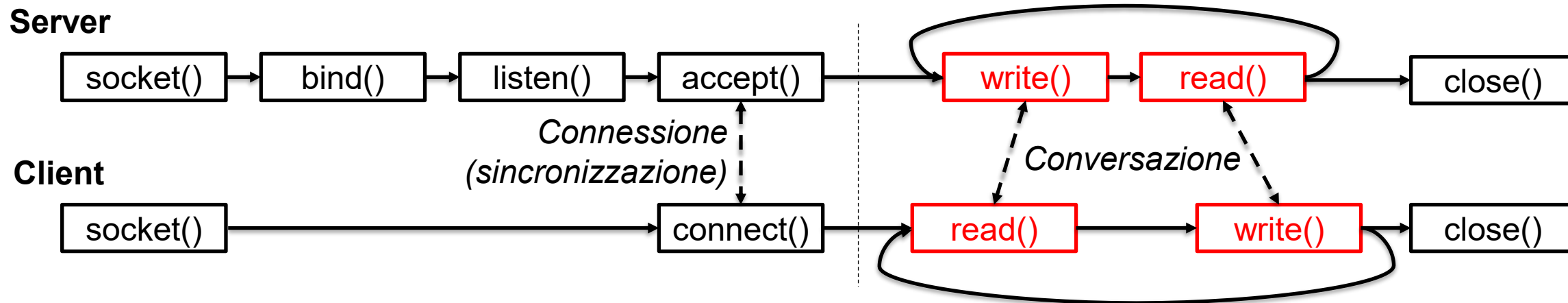
Interazione client/server: TCP (pattern di comunicazione ed API)

Server



- **Importante!** Le chiamate di sistema **accept()**, **connect()** e **read()** sono bloccanti!
 - **connect()** blocca l'esecuzione finché non è finalizzato il three-way handshake (o evento di errore)
 - **accept()** blocca l'esecuzione finché non è finalizzato il three-way handshake (o evento di errore) a seguito di una richiesta di connessione
 - **read()** blocca l'esecuzione finché non ci sono dati da prelevare nel buffer di ricezione (o evento di errore)
- Le altre chiamate di sistema **non** sono bloccanti
 - Possibile eccezione: **write()** → Può bloccare se il buffer di invio è pieno

Interazione client/server: TCP (pattern di comunicazione ed API)



- **Domanda:** è sempre necessario seguire il pattern di comunicazione specificato con TCP?
 - **Sì** per la fase di connessione (**modello client-server**)
 - **No** per la fase di conversazione
 - **Non** è necessario seguire un **modello client-server**
 - Non per forza il client deve inviare richieste e il server risposte
- In fase di conversazione TCP è stato definito per trasferire **flussi di byte** bidirezionalmente secondo un modello **peer-to-peer**
 - La semantica dei dati è definita dall'applicazione
- **Esempio:** `read()` e `write()` possono essere invertite

Berkeley Sockets: riassunto

Primitiva / Chiamata di sistema	Significato	Utilizzo tipico
socket()	Crea un nuovo endpoint per la comunicazione	UDP, TCP
bind()	Associa un indirizzo locale alla socket	UDP, TCP
listen()	Annuncia l'intenzione di ricevere richieste di connessione	TCP
accept()	Blocca il chiamante finché non arriva una richiesta di connessione	TCP
connect()	Tenta attivamente di stabilire una connessione	TCP
write()	Invia dati per mezzo della connessione	TCP
read()	Riceve dati per mezzo della connessione	TCP
sendto()	Invia dati	UDP
recvfrom()	Riceve dati	UDP
close()	Chiude la socket	UDP, TCP

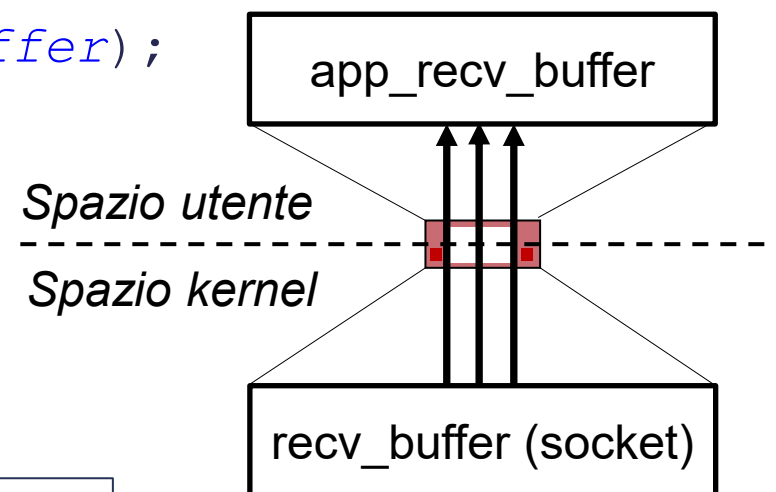
read(): accorgimenti

- Le socket TCP trasportano flussi di byte, quindi
 - Non si ha il concetto di «messaggio» (il flusso è continuo, potenzialmente senza fine)
 - La lettura da parte del processo applicativo avviene per un numero arbitrario di byte
 - I bytes vengono rimossi dal buffer di ricezione una volta letti
- Il prototipo (in pseudocodice) della **read()** è quindi:

```
byteLetti read(socket, app_recv_buffer, dimBuffer);
```

```
byteLetti      = byte effettivamente letti dalla socket  
socket         = il canale da cui leggere  
app_recv_buffer = lo spazio di memoria RAM destinato all'applicazione  
                 dove trasferire i byte letti (ricevuti)  
                 = dimensione dell'app_buffer (numero max di caratteri  
                 che si possono leggere dalla socket)
```

L'applicazione deve prevedere **cicli di lettura** dal buffer di ricezione!





Le socket in Java (TCP)

Le socket in Java

- Java definisce alcune classi che costituiscono un'interfaccia ad oggetti alle system call illustrate in precedenza
- Le principali sono:
 - `java.net.Socket` (lato client)
 - `java.net.ServerSocket` (lato server)
- Queste classi accorpano funzionalità e mascherano alcuni dettagli con il vantaggio di semplificarne l'uso
- Necessario conoscerne il funzionamento per poterle utilizzare in modo efficace
 - Le prossime slide discutono i costruttori e i principali metodi delle due classi

Riferimenti principali:

<https://docs.oracle.com/en/java/javase/13/docs/api/java.base/java/net/Socket.html>

<https://docs.oracle.com/en/java/javase/13/docs/api/java.base/java/net/ServerSocket.html>

java.net.Socket - Costruttori

- **public Socket()**
 - Crea una socket per il client non connessa
- **public Socket(String host, int port)**
throws UnknownHostException, IOException
 - Crea una socket e la connette al numero di porta remoto specificato sull'hostname remoto specificato
 - Se l'host non è specificato si considera l'indirizzo sull'interfaccia di loopback
 - L'eccezione `UnknownHostException` viene lanciata se l'indirizzo IP dell'host non può essere determinato
- **public Socket(InetAddress address, int port)**
throws IOException
 - Crea una socket e la connette al numero di porta remoto specificato sull'host con indirizzo IP remoto specificato

java.net.Socket - Metodi per la gestione delle connessioni

- **public void bind(SocketAddress bindpoint) throws IOException**
 - Associa la socket a un indirizzo locale (indirizzo IP e porta)
 - Se l'indirizzo è `null`, il sistema sceglierà una porta effimera e un indirizzo IP locale valido
- **public void connect(SocketAddress endpoint) throws IOException**
 - Connette la socket all'endpoint specificato (indirizzo IP e porta del server)
- **public void connect(SocketAddress endpoint, int timeout) throws IOException**
 - Collega la socket all'endpoint specificato settando un timeout (in millisecondi)
- **public void close()**
 - Chiude la socket

java.net.Socket - Metodi per lo scambio di byte

- **public InputStream getInputStream() throws IOException**
 - Restituisce uno stream di dati (byte grezzi) in input dalla socket
- **public OutputStream getOutputStream() throws IOException**
 - Restituisce un output stream per scrivere byte grezzi sulla socket

java.net.ServerSocket - Costruttori

- **public ServerSocket() throws IOException**
 - Crea una socket per il server non associata ad alcuna porta
- **public ServerSocket(int port) throws IOException**
 - Crea una socket per il server associata alla porta specificata
 - Una porta pari a zero crea una socket su una porta effimera libera qualsiasi
 - La lunghezza massima della coda (backlog) per le richieste di connessione complete in entrata (Completed Connection Queue) è impostata a 50
- **public ServerSocket(int port, int backlog) throws IOException**
 - Permette di specificare esplicitamente anche la dimensione della Completed Connection Queue (backlog)

java.net.ServerSocket - Metodi per la gestione delle connessioni

- **public void bind(SocketAddress endpoint) throws IOException**
 - Associa la socket a un indirizzo locale (indirizzo IP e porta)
 - Se l'indirizzo è `null`, il sistema sceglierà una porta effimera e un indirizzo IP locale valido
- **public Socket accept() throws IOException**
 - Resta in ascolto di una connessione e l'accetta
 - Restituisce una nuova socket (socket di servizio)
 - Il metodo rimane bloccato finché non viene stabilita una connessione
- Altri metodi di utility
 - **public InetAddress getInetAddress()**
 - Restituisce l'indirizzo locale della socket, oppure `null` se la socket non è associata a un indirizzo
 - **public int getLocalPort()**
 - Restituisce la porta sulla quale la socket è in ascolto, oppure -1 se non è ancora associata
 - **public SocketAddress getLocalSocketAddress()**
 - Restituisce l'indirizzo dell'endpoint a cui la socket è associata, oppure `null` se non è ancora associata

Esempio pratico (EsempioClientServer)

- Vediamo un esempio dove:
 - Un server accetta una connessione da un client, quindi invia un flusso di byte (ad esempio una stringa di caratteri) al client
 - Un client richiede una connessione, quindi legge un flusso di byte inviato dal server

- L'obiettivo è dimostrare che:
 - Il comportamento dei processi coinvolti è indipendente l'uno dall'altro
 - I processi si scambiano flussi di byte
 - Il modo di gestire i dati da parte delle socket è estraneo alla nozione di «messaggio» o di «protocollo applicativo»

Codice del server - SenderServerSocket (1)

```
1. import java.io.PrintWriter;
2. import java.net.ServerSocket;
3. import java.net.Socket;

4. public class SenderServerSocket {

5.     final static String message =
6.         "This is a not so short text to test the reading capabilities of clients.";

7.     public static void main(String[] args) {

8.         try {
9.             Socket clientSocket;
10.            ServerSocket listenSocket;

11.            listenSocket = new ServerSocket(53535);
12.            System.out.println("Running Server:" +
13.                " Host= " + listenSocket.getInetAddress() +
14.                " Port= " + listenSocket.getLocalPort());
```

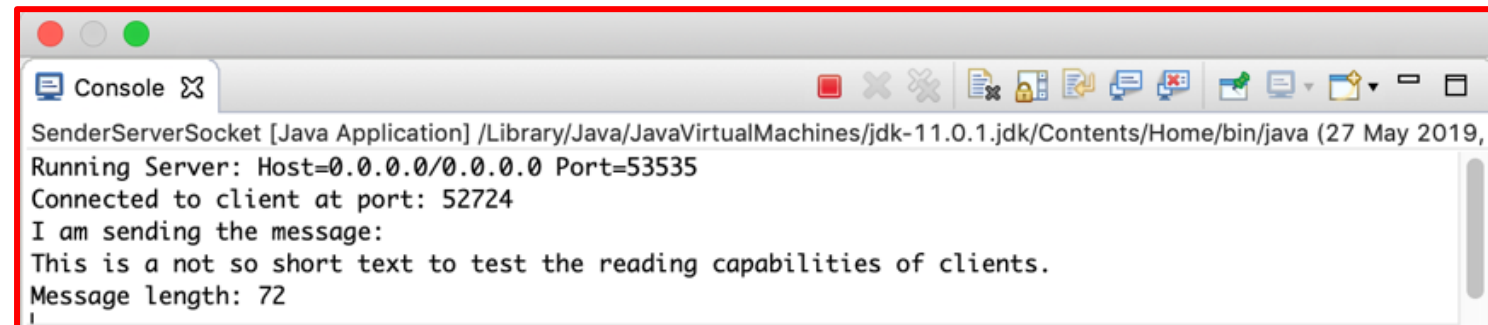
Codice del server - SenderServerSocket (2)

```
15. // ciclo infinito per aprire una socket di servizio, inviare il messaggio, chiudere la socket
    di servizio
16. while (true) {
17.     clientSocket = listenSocket.accept();
18.     System.out.println("Connected to client at port: " + clientSocket.getPort());
19.     PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);

20.     System.out.println("I am sending the message: \n" + message);
21.     System.out.println("message length: " + message.length());

22.     out.write(message);
23.     out.flush();

24.     clientSocket.close();
25. }
26. } catch (Exception e) {
27.     e.printStackTrace();
28. }
29. }
30. }
```



```
SenderServerSocket [Java Application] /Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java (27 May 2019,
Running Server: Host=0.0.0.0/0.0.0.0 Port=53535
Connected to client at port: 52724
I am sending the message:
This is a not so short text to test the reading capabilities of clients.
Message length: 72
```


Codice del server - SenderServerSocket - Dettagli

```
9.  Socket clientSocket;  
10. ServerSocket listenSocket;
```

■ Dichiarazione delle variabili

- `clientSocket` → Socket di servizio del tipo `Socket`
- `listenSocket` → Socket di ascolto del tipo `ServerSocket`

```
11. listenSocket = new ServerSocket(53535);
```

■ Creazione dell'oggetto relativo alla socket di ascolto

- Associazione (bind) alla porta 53535
- Creazione del backlog delle connessioni complete (Completed Connection Queue)

■ **Nota:** il server si mette in ascolto su tutte le interfacce di rete disponibili, inclusa l'interfaccia di loopback (indirizzo speciale 0.0.0.0)

```
17. clientSocket = listenSocket.accept();
```

■ Creazione dell'oggetto relativo alla socket di servizio, non appena una connessione è stata stabilita (connessione estratta da Completed Connection Queue)

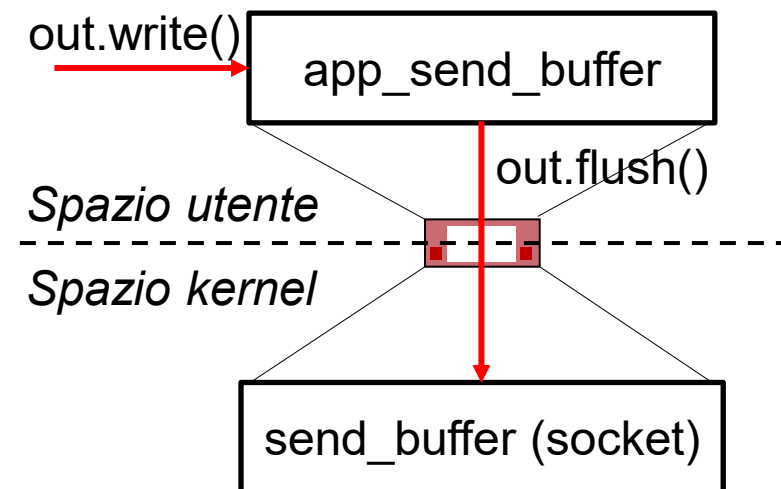
Codice del server - SenderServerSocket - Dettagli

```
19. PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
```

- Creazione dell'oggetto `out` del tipo `PrintWriter` per tradurre comodamente stringhe di testo nei byte da inviare attraverso la socket

```
22. out.write(message);  
23. out.flush();
```

- Con `write()` viene copiato il contenuto della stringa `message` (convertito in byte) nel buffer dell'applicazione (`app_send_buffer`)
- Con `flush()` i byte vengono passati al buffer di invio nello spazio kernel (`send_buffer`) per l'invio immediato



```
24. clientSocket.close();
```

- Chiusura della socket
 - Una volta chiusa, si ricomincia il ciclo `while (true) { }`

Codice del client - ReceiverClientSocket (1)

```
1. import java.io.DataInputStream;
2. import java.io.IOException;
3. import java.net.InetAddress;
4. import java.net.Socket;

5. public class ReceiverClientSocket {

6.     public static void main(String[] args) {
7.         Socket socket; // la mia socket
8.         InetAddress serverAddress; // l'indirizzo del server
9.         int serverPort; // la porta del server

10.        try { // connessione al server
11.            serverAddress = InetAddress.getByName(args[0]);
12.            serverPort = Integer.parseInt(args[1]);
13.            socket = new Socket(serverAddress, serverPort);

14.            System.out.println("Connected to: " + socket.getInetAddress());
```

Codice del client - ReceiverClientSocket (2)

```
15.    DataInputStream in; // sorgente del flusso di byte
16.    byte[] byteReceived = new byte[1000]; // buffer temporaneo
17.    String messageString = ""; // testo da mostrare a schermo

18.    // flusso da cui leggere
19.    in = new DataInputStream(socket.getInputStream());
20.    System.out.println("Ready to read from the socket");

21.    // il codice seguente mostra in dettaglio come leggere dalla socket TCP
22.    int bytesRead = 0; // numero di byte letti
23.    bytesRead = in.read(byteReceived);
24.    messageString += new String(byteReceived, 0, bytesRead);

25.    System.out.println("Received: " + messageString);
26.    System.out.println("I am done!");

27.    socket.close();
28.    } catch (IOException e) {
29.        e.printStackTrace();
30.    }
31. }
32. }
```

Codice del client - ReceiverClientSocket - Dettagli

```
11.  serverAddress = InetAddress.getByName(args[0]);  
12.  serverPort = Integer.parseInt(args[1]);  
13.  socket = new Socket(serverAddress, serverPort);
```

- Indirizzo IP e porta del server sono presi in ingresso da riga di comando (`args[0]` e `args[1]`)
- Creazione dell'oggetto relativo alla socket del client

```
16.  byte[] byteReceived = new byte[1000];
```

- Istanziamento del buffer locale dell'applicazione (dimensione di 1000 byte)
- Se il server invia un messaggio più lungo di 1000 byte, il resto dei dati rimarrà nel buffer di ricezione nello spazio kernel in attesa di un'altra lettura

```
19.  in = new DataInputStream(socket.getInputStream());
```

- Creazione dell'oggetto `in` del tipo `DataInputStream` per leggere i tipi di dati primitivi di Java (come `int`, `float`, `long`, `boolean`) partendo dai byte

Codice del client - ReceiverClientSocket - Dettagli

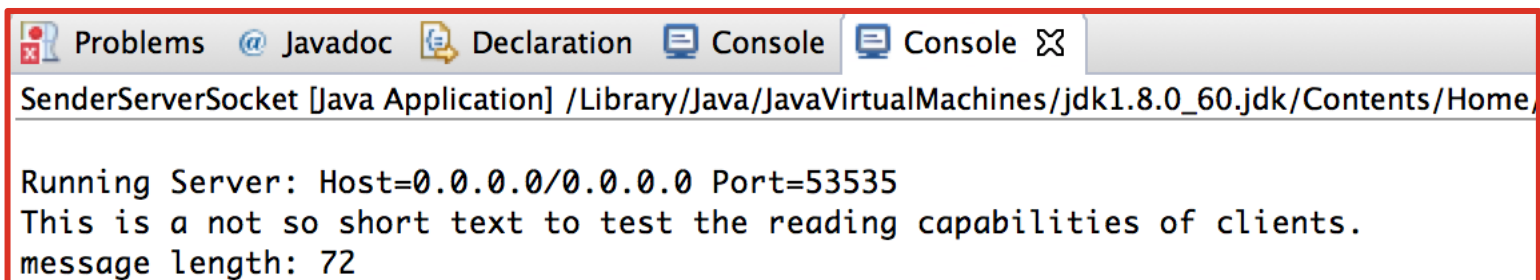
```
23.  bytesRead = in.read(byteReceived);  
24.  messageString += new String(byteReceived, 0, bytesRead);
```

- Viene prelevato il contenuto del buffer in ricezione e viene riempito il buffer dell'applicazione `byteReceived`
- I byte letti vengono convertiti in stringa per la porzione ricevuta (da 0 a `bytesRead`)

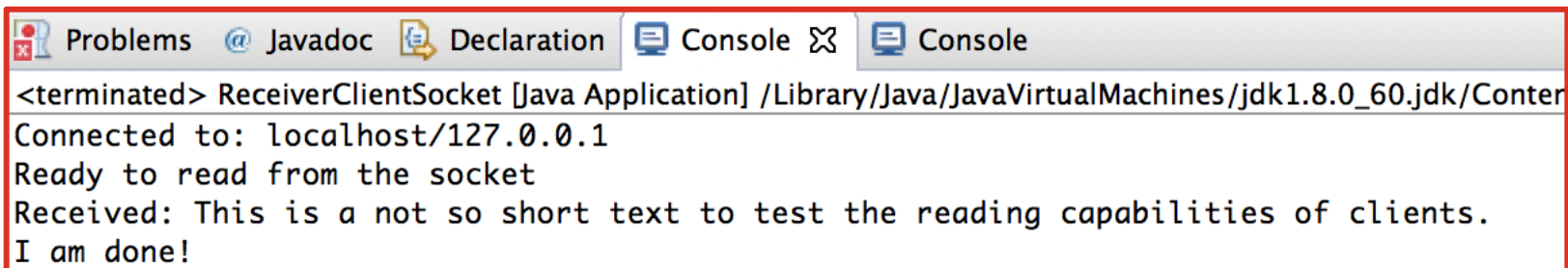
```
27.  socket.close();
```

- Chiusura della socket

Esecuzione



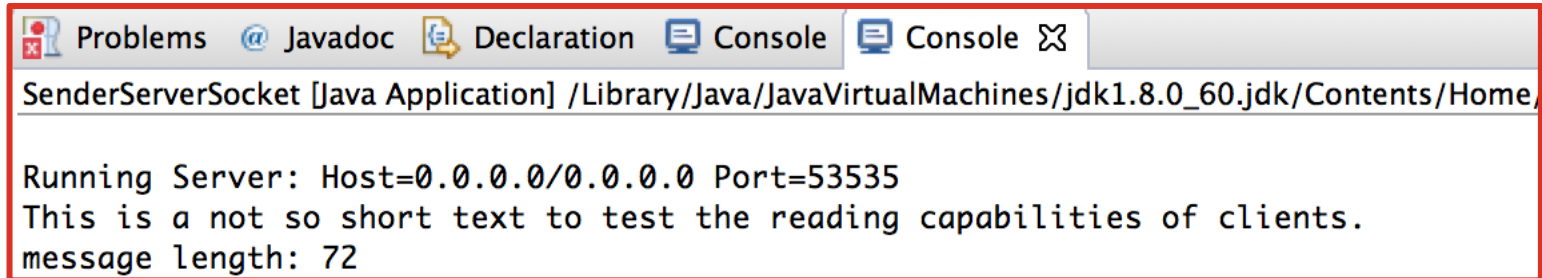
```
SenderServerSocket [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_60.jdk/Contents/Home,  
  
Running Server: Host=0.0.0.0/0.0.0.0 Port=53535  
This is a not so short text to test the reading capabilities of clients.  
message length: 72
```



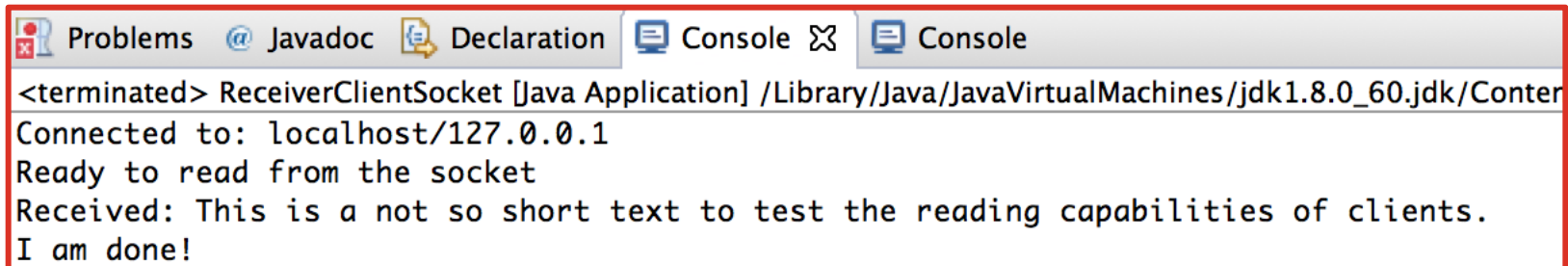
```
<terminated> ReceiverClientSocket [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_60.jdk/Conter  
Connected to: localhost/127.0.0.1  
Ready to read from the socket  
Received: This is a not so short text to test the reading capabilities of clients.  
I am done!
```

- Funziona!
- Il codice del client però non è corretto, anche se tutto sembra funzionare... Perché?

Esecuzione



```
SenderServerSocket [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_60.jdk/Contents/Home,  
  
Running Server: Host=0.0.0.0/0.0.0.0 Port=53535  
This is a not so short text to test the reading capabilities of clients.  
message length: 72
```



```
<terminated> ReceiverClientSocket [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_60.jdk/Conter  
Connected to: localhost/127.0.0.1  
Ready to read from the socket  
Received: This is a not so short text to test the reading capabilities of clients.  
I am done!
```

- Funziona!
- Il codice del client però non è corretto, anche se tutto sembra funzionare... Perché?
- Per capirlo introduciamo un **Lazy Server**
 - Server che invia pochi byte alla volta con un ritardo tra un invio e l'altro

Codice del server - LazySenderServerSocket

```
14. ...
15. // ciclo infinito per aprire una socket di servizio, inviare il messaggio, chiudere la socket
    di servizio
16. while (true) {
17.     clientSocket = listenSocket.accept();

18.     PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
19.     System.out.println("message length: " + message.length());

20.     final int chunk = 9; // numero di byte inviati ogni volta
21.     boolean end = false;

22.     int i;
23.     for (i = 0; i < message.length() - chunk; i += chunk) {
24.         System.out.println(message.substring(i, i + chunk));
25.         Thread.sleep(1000); // lazy server: aspetta 1" prima di inviare
26.         out.write(message.substring(i, i + chunk));
27.         out.flush();
28.     }
29.     System.out.println(message.substring(i, message.length()));
30.     out.write(message.substring(i, message.length()));
31.     out.flush();

32.     clientSocket.close();
33. }
34. ...
```

Codice del server - LazySenderServerSocket

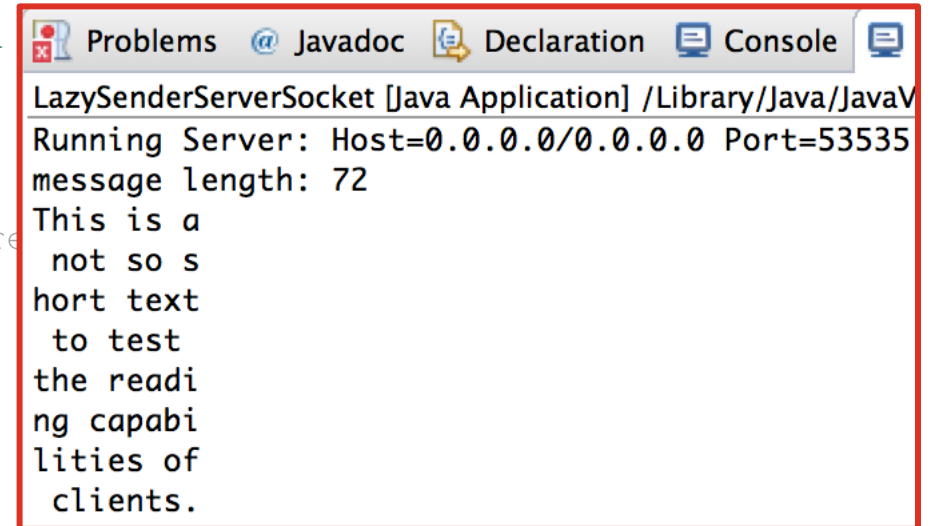
```
14. ....
15. // ciclo infinito per aprire una socket di servizio, inviare il
    di servizio
16. while (true) {
17.     clientSocket = listenSocket.accept();

18.     PrintWriter out = new PrintWriter(clientSocket.getOutputStream());
19.     System.out.println("message length: " + message.length());

20.     final int chunk = 9; // numero di byte inviati ogni volta
21.     boolean end = false;

22.     int i;
23.     for (i = 0; i < message.length() - chunk; i += chunk) {
24.         System.out.println(message.substring(i, i + chunk));
25.         Thread.sleep(1000); // lazy server: aspetta 1" prima di inviare
26.         out.write(message.substring(i, i + chunk));
27.         out.flush();
28.     }
29.     System.out.println(message.substring(i, message.length()));
30.     out.write(message.substring(i, message.length()));
31.     out.flush();

32.     clientSocket.close();
33. }
34. ....
```



LazySenderServerSocket - Comportamento del client (ReceiverClientSocket)

```
15.    DataInputStream in; // sorgente del flusso di byte
16.    byte[] byteReceived = new byte[1000]; // buffer temporaneo
17.    String messageString = ""; // testo da mostrare a schermo

18.    // flusso da cui leggere
19.    in = new DataInputStream(socket.getInputStream());
20.    System.out.println("Ready to read from the socket");

21.    // il codice seguente mostra in dettaglio come leggere dalla socket TCP
22.    int bytesRead = 0; // numero di byte letti
23.    bytesRead = in.read(byteReceived);
24.    messageString += new String(byteReceived, 0, bytesRead);

25.    System.out.println("Received: " + messageString);
26.    System.out.println("I am done!");

27.    socket.close();
28.    } catch (IOException e) {
29.        e.printStackTrace();
30.    }
31. }
32. }
```

LazySenderServerSocket - Comportamento del client (ReceiverClientSocket)

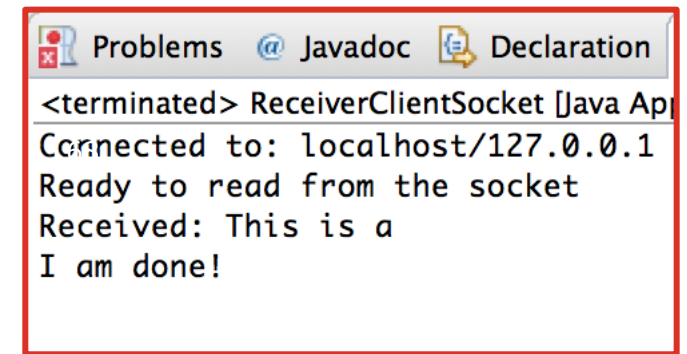
```
15.    DataInputStream in; // sorgente del flusso di byte
16.    byte[] byteReceived = new byte[1000]; // buffer temporaneo
17.    String messageString = ""; // testo da mostrare a schermo

18.    // flusso da cui leggere
19.    in = new DataInputStream(socket.getInputStream());
20.    System.out.println("Ready to read from the socket");

21.    // il codice seguente mostra in dettaglio come leggere dalla socket TCP
22.    int bytesRead = 0; // numero di byte letti
23.    bytesRead = in.read(byteReceived);
24.    messageString += new String(byteReceived, 0, bytesRead);

25.    System.out.println("Received: " + messageString);
26.    System.out.println("I am done!");

27.    socket.close();
28.    } catch (IOException e) {
29.        e.printStackTrace();
30.    }
31. }
32. }
```



LazySenderServerSocket - Comportamento del client (ReceiverClientSocket)

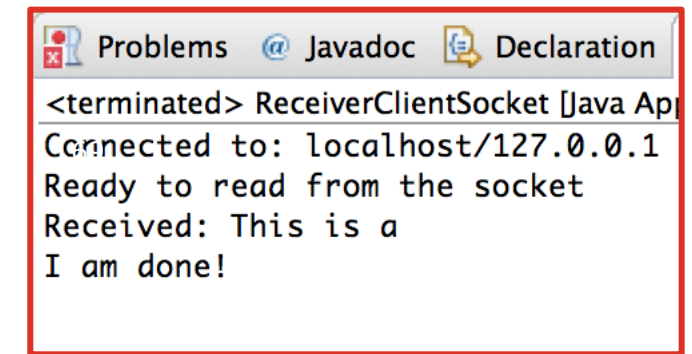
```
15.    DataInputStream in; // sorgente del flusso di byte
16.    byte[] byteReceived = new byte[1000]; // buffer temporaneo
17.    String messageString = ""; // testo da mostrare a schermo

18.    // flusso da cui leggere
19.    in = new DataInputStream(socket.getInputStream());
20.    System.out.println("Ready to read from the socket");

21.    // il codice seguente mostra in dettaglio come leggere dalla socket TCP
22.    int bytesRead = 0; // numero di byte letti
23.    bytesRead = in.read(byteReceived);
24.    messageString += new String(byteReceived, 0, bytesRead);

25.    System.out.println("Received: " + messageString);
26.    System.out.println("I am done!");

27.    socket.close();
28.    } catch (IOException e) {
29.        e.printStackTrace();
30.    }
31. }
32. }
```



Domanda: cosa non funziona?

Codice del client modificato - SmartReceiverClientSocket

```
15.    DataInputStream in; // sorgente del flusso di byte
16.    byte[] byteReceived = new byte[1000]; // buffer temporaneo
17.    String messageString = ""; // testo da mostrare a schermo
18.    // flusso da cui leggere
19.    in = new DataInputStream(socket.getInputStream());
20.    System.out.println("Ready to read from the socket");
21.    // The following code shows in detail how to read from a TCP socket
22.    int bytesRead = 0; // the number of bytes read
23.    while (true) {
24.        bytesRead = in.read(byteReceived);
25.        if (bytesRead == -1)
26.            break; // non ci sono altri byte
27.        messageString += new String(byteReceived, 0, bytesRead);
28.        System.out.println("Received: " + messageString);
29.    }
30.    System.out.println("I am done!");
31.    socket.close();
32. } catch (IOException e) {
33.     e.printStackTrace();
34. }
35. }
36. }
```

Codice del client modificato - SmartReceiverClientSocket

```
15.    DataInputStream in; // sorgente del flusso di byte
16.    byte[] byteReceived = new byte[1000]; // buffer temporaneo
17.    String messageString = ""; // testo da mostrare a schermo
18.    // flusso da cui leggere
19.    in = new DataInputStream(socket.getInputStream());
20.    System.out.println("Ready to read from the socket");
21.    // The following code shows in detail how to read from a TCP socket
22.    int bytesRead = 0; // the number of bytes read
23.    while (true) {
24.        bytesRead = in.read(byteReceived);
25.        if (bytesRead == -1)
26.            break; // non ci sono altri byte
27.        messageString += new String(byteReceived, 0, bytesRead);
28.        System.out.println("Received: " + messageString);
29.    }
30.    System.out.println("I am done!");
31.    socket.close();
32. } catch (IOException e) {
33.     e.printStackTrace();
34. }
35. }
36. }
```

I dati provenienti dalle socket
devono sempre essere letti da
un ciclo di lettura

Codice del client modificato - SmartReceiverClientSocket

```
15.    DataInputStream in; // sorgente del flusso di byte
16.    byte[] byteReceived = new byte[1000]; // buffer temporaneo
17.    String messageString = ""; // testo da mostrare a schermo
18.    // flusso da cui leggere
19.    in = new DataInputStream(socket.getInputStream());
20.    System.out.println("Ready to read from the socket");
21.    // The following code shows in detail how to read from a TCP socket
22.    int bytesRead = 0; // the number of bytes read
23.    while (true) {
24.        bytesRead = in.read(byteReceived);
25.        if (bytesRead == -1)
26.            break; // non ci sono altri byte
27.        messageString += new String(byteReceived, 0, bytesRead);
28.        System.out.println("Received: " + messageString);
29.    }
30.    System.out.println("I am done!");
31.    socket.close();
32. } catch (IOException e) {
33.     e.printStackTrace();
34. }
35. }
36. }
```

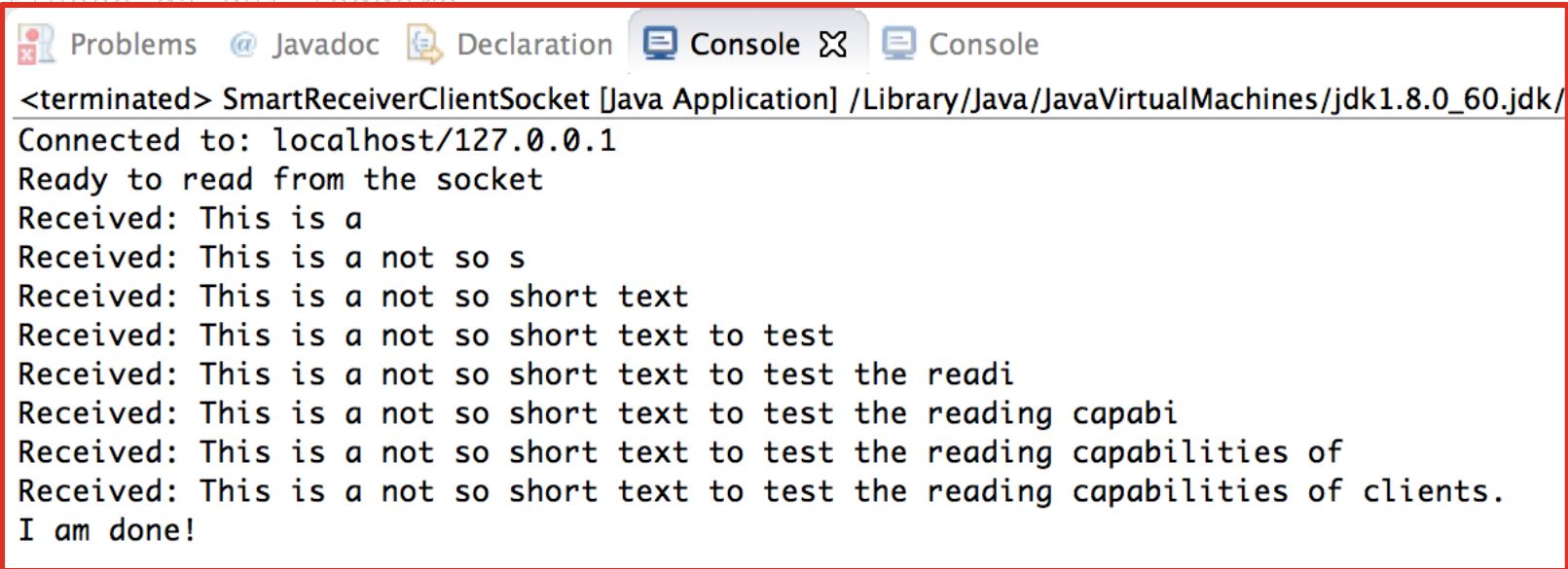

Codice del client modificato - SmartReceiverClientSocket

```
15.    DataInputStream in; // sorgente del flusso di byte
16.    byte[] byteReceived = new byte[1000]; // buffer temporaneo
17.    String messageString = ""; // testo da mostrare a schermo
18.    // flusso da cui leggere
19.    in = new DataInputStream(socket.getInputStream());
20.    System.out.println("Ready to read from the socket");
21.    // The following code shows in detail how to read from a TCP socket
22.    int bytesRead = 0; // the number of bytes read
23.    while (true) {
24.        bytesRead = in.read(byteReceived);
25.        if (bytesRead == -1)
26.            break; // non ci sono altri byte
27.        messageString += new String(byteReceived, 0, bytesRead);
28.        System.out.println("Received: " + messageString);
29.    }
30.    System.out.println("I am done!");
31.    socket.close();
32. } catch (IOException e) {
33.     e.printStackTrace();
34. }
35. }
36. }
```

Quando `read()` restituisce -1
significa che il server ha chiuso
la connessione (ha chiamato
`socket.close()`)

Codice del client modificato - SmartReceiverClientSocket

```
15.   DataInputStream in; // sorgente del flusso di byte
16.   byte[] byteReceived = new byte[1000]; // buffer temporaneo
17.   String messageString = ""; // testo da mostrare a schermo
18.   // flusso da cui leggere
19.   in = new DataInputStream(socket);
20.   System.out.println("Connected to: localhost/127.0.0.1");
21.   // Ready to read from the socket
22.   while (true) {
23.       int b;
24.       while (b != -1) {
25.           b = in.read();
26.           m = new String(byteReceived);
27.           m += (char) b;
28.           System.out.println("Received: " + m);
29.       }
30.       System.out.println("I am done!");
31.       socket.close();
32.   } catch (IOException e) {
33.       e.printStackTrace();
34.   }
35. }
36. }
```



```
<terminated> SmartReceiverClientSocket [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_60.jdk/
Connected to: localhost/127.0.0.1
Ready to read from the socket
Received: This is a
Received: This is a not so s
Received: This is a not so short text
Received: This is a not so short text to test
Received: This is a not so short text to test the readi
Received: This is a not so short text to test the reading capabi
Received: This is a not so short text to test the reading capabilities of
Received: This is a not so short text to test the reading capabilities of clients.
I am done!
```

Codice del client modificato - SmartReceiverClientSocket

```

15.    DataInputStream in; // sorgente del flusso di byte
16.    byte[] byteReceived = new byte[1000]; // buffer temporaneo
17.    String messageString = ""; // testo da mostrare a schermo
18.    // flusso da cui leggere
19.    in = new DataInputStream(socket);
20.    System.out.println("Connected to: localhost/127.0.0.1");
21.    // Ready to read from the socket
22.    while (true) {
23.        int n = in.read(byteReceived);
24.        String s = new String(byteReceived, 0, n);
25.        System.out.println("Received: " + s);
26.        if (s.equals("I am done!")) {
27.            break;
28.        }
29.    }
30.    System.out.println("I am done!");
31.    socket.close();
32.    } catch (IOException e) {
33.        e.printStackTrace();
34.    }
35. }
36. }

```

Problems Javadoc Declaration Console Console

```

<terminated> SmartReceiverClientSocket [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_60.jdk/
Connected to: localhost/127.0.0.1
Ready to read from the socket
Received: This is a
Received: This is a not so s
Received: This is a not so short text
Received: This is a not so short text to test
Received: This is a not so short text to test the readi
Received: This is a not so short text to test the reading capabi
Received: This is a not so short text to test the reading capabilities of
Received: This is a not so short text to test the reading capabilities of clients.
I am done!

```

Ora funziona davvero!

Progettare un'applicazione con le socket: tiriamo le somme

- **Client:** L'architettura è concettualmente più semplice di quella di un server
 - È spesso un'applicazione convenzionale che legge/scrive da una socket anziché da un altro canale I/O

- **Server:** L'architettura generale prevede che
 - Venga creata una socket con una porta nota per accettare le richieste di connessione
 - Entri in un ciclo infinito in cui alternare
 1. Attesa/accettazione di una richiesta di connessione da un client
 2. Ciclo {lettura dati, esecuzione, invio dati} fino al termine della conversazione
 - Durata spesso stabilita dal client
 3. Chiusura della connessione

Progettare un'applicazione con le socket: Java vs. C (cenni)

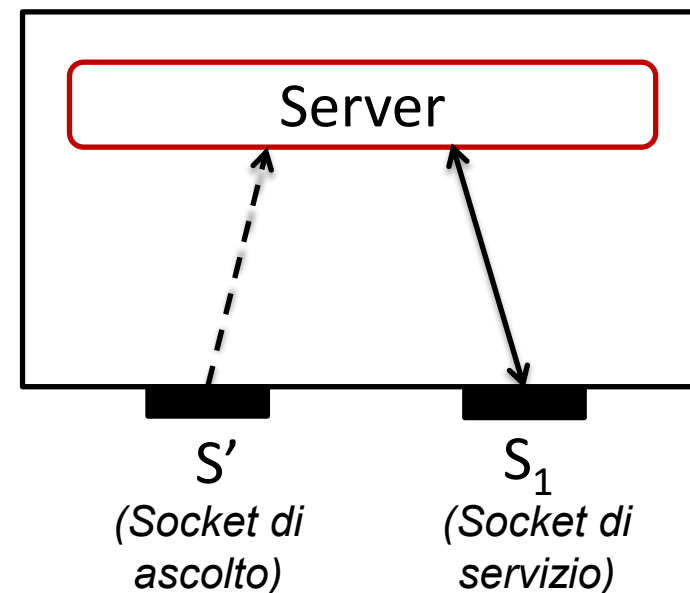
- Java mette a disposizione classi e metodi che “mascherano” le chiamate di sistema
- Esempio: `listenSocket = new ServerSocket(53535);`
 - “Sotto il cofano” vengono effettuate le chiamate di sistema **socket()**, **bind()**, **listen()**
- In C viene usata la libreria `sys/socket.h` che include esattamente le Berkeley sockets
- I pattern di comunicazione visti in precedenza devono essere seguiti alla lettera
 - Es. si avrà una riga di codice per chiamare una funzione della libreria `sys/socket.h` che effettuerà la **listen()**
 - La funzione stessa si chiama `listen`



Architetture server

Tipi di server - Server iterativo

- Finora ci siamo implicitamente soffermati sull'implementazione di un **server iterativo**
 - `SenderServerSocket` e `LazySenderServerSocket` sono esempi di server iterativi
- Al momento di una richiesta di connessione sulla **socket di ascolto**, il server crea una **socket di servizio** per effettuare una conversazione con il client
- Le eventuali ulteriori richieste di connessione per il server verranno accodate alla porta nota per essere successivamente soddisfatte
 - Servite solo a seguito di **`close()`** della socket di servizio corrente da parte del server
- Vantaggi:
 - Semplice da progettare
- Svantaggi:
 - Viene servito un client alla volta, gli altri devono attendere
 - Un client può impedire l'evoluzione di altri client
 - Non scala
- Soluzione: **server concorrenti**



Da server iterativo a server concorrente

Legenda:

—→ Connessione
—→ Conversazione



server (*serverIP*)

crea socket su numero di porta x (**listening socket**, o **socket di ascolto**)

attendi una richiesta di connessione in ingresso, quindi crea una nuova socket (**connection socket**, o **socket di servizio**)

ricevi la richiesta da **connection socket**

invia la risposta via **connection socket** (destinazione *clientIP* e porta client)

chiudi **connection socket**

setup connessione TCP



client (*clientIP*)

crea socket (**client socket**)

crea richiesta di connessione con indirizzo di destinazione *serverIP* e numero di porta x

invia la richiesta via **client socket** (destinazione *serverIP* e porta x)

ricevi la risposta da **client socket**

chiudi **client socket**

richiesta

risposta

Da server iterativo a server concorrente

Legenda:

—→ Connessione
—→ Conversazione



server (*serverIP*)

crea socket su numero di porta x (**listening socket**, o **socket di ascolto**)

attendi una richiesta di connessione in ingresso, quindi crea una nuova socket (**connection socket**, o **socket di servizio**)

ricevi la richiesta da **connection socket**

invia la risposta via **connection socket** (destinazione *clientIP* e porta client)

chiudi **connection socket**



client (*clientIP*)

crea socket (**client socket**)

crea richiesta di connessione con indirizzo di destinazione *serverIP* e numero di porta x

invia la richiesta via **client socket** (destinazione *serverIP* e porta x)

ricevi la risposta da **client socket**

chiudi **client socket**

setup connessione TCP

richiesta

risposta

Da server iterativo a server concorrente

Legenda:

—→ Connessione
—→ Conversazione



server (*serverIP*)

crea socket su numero di porta x (**listening socket**, o **socket di ascolto**)

attendi una richiesta di connessione in ingresso, quindi crea una nuova socket (**connection socket**, o **socket di servizio**)

ricevi la richiesta da **connection socket**

invia la risposta via **connection socket** (destinazione *clientIP* e porta client)

chiudi **connection socket**



client (*clientIP*)

crea socket (**client socket**)

crea richiesta di connessione con indirizzo di destinazione *serverIP* e numero di porta x

invia la richiesta via **client socket** (destinazione *serverIP* e porta x)

ricevi la risposta da **client socket**

chiudi **client socket**

setup connessione TCP

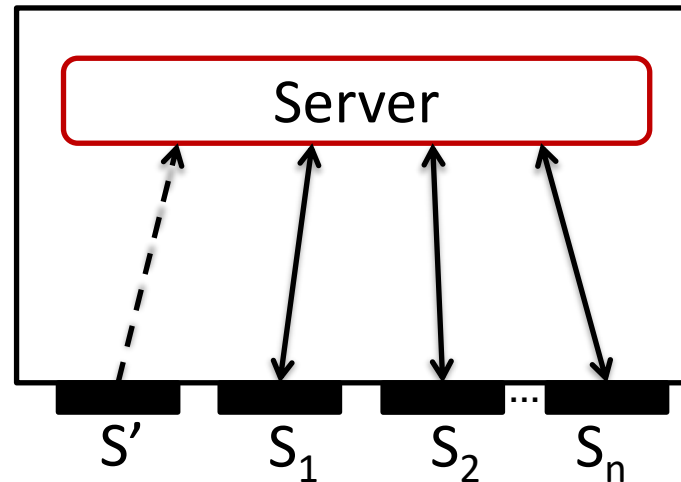
richiesta

risposta

Tipi di server - Server concorrente

- Tre tipi di server concorrente:
 - **A processo singolo:** “simula” la presenza di un server dedicato per ogni connessione stabilita con un client, ma mantenendo un singolo processo
 - **Multi-processo:** si ha un processo dedicato per ogni connessione stabilita con un client
 - **Multi-thread:** si ha un thread dedicato per ogni connessione stabilita con un client, mantenendo un singolo processo

Tipi di server - Server concorrente a processo singolo



- Viene utilizzata una chiamata di sistema specifica: **select()**
 - Monitora un insieme di socket e avvisa il programma non appena una di esse è pronta per un'operazione (lettura, scrittura, errore)
 - Prende anche il nome di architettura basata su **I/O multiplexing**
- Si evitano due situazioni:
 - La necessità di effettuare un'interrogazione (**polling**) a rotazione per ogni canale I/O (socket), sprestando CPU
 - Rimanere bloccati su una chiamata di **read()** o **write()** quando non ci sono dati da leggere o scrivere sulla socket specifica

Tipi di server - Server concorrente a processo singolo

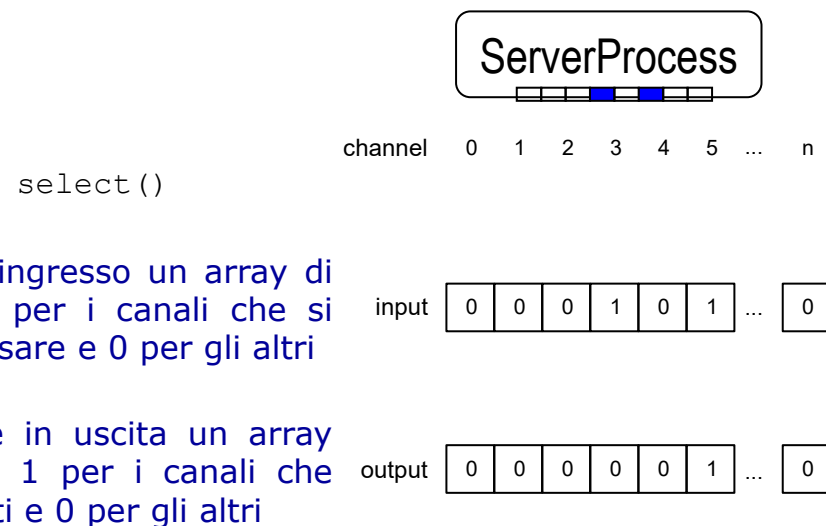
- La **select()** permette di leggere e scrivere da **canali I/O** in maniera **non bloccante**

- Il processo viene sospeso finché non è possibile fare una operazione di I/O

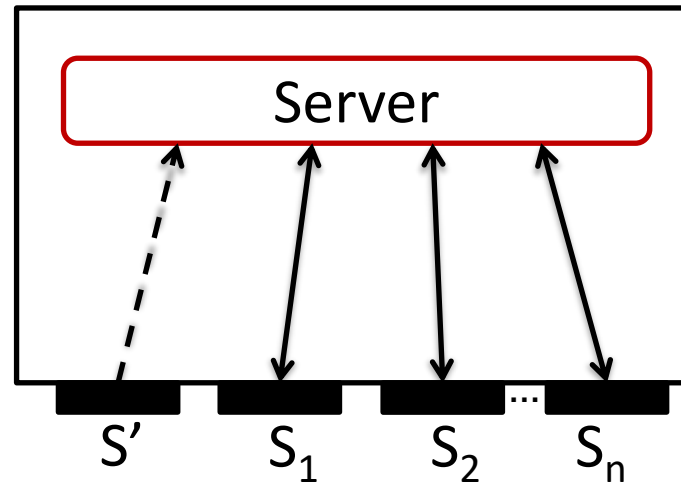
- Il codice deve essere quindi strutturato in questo modo:

1. Dico al sistema quali canali voglio usare in modalità non bloccante
2. Chiamo la **select()** che controlla quali canali sono «pronti»
3. Sui canali pronti effettuo l'operazione **read()** o **write()** desiderata
4. Ciclo tornando al punto 1

- Questo garantisce che se viene effettuata un'operazione di lettura/scrittura il controllo viene restituito al processo immediatamente



Tipi di server - Server concorrente a processo singolo



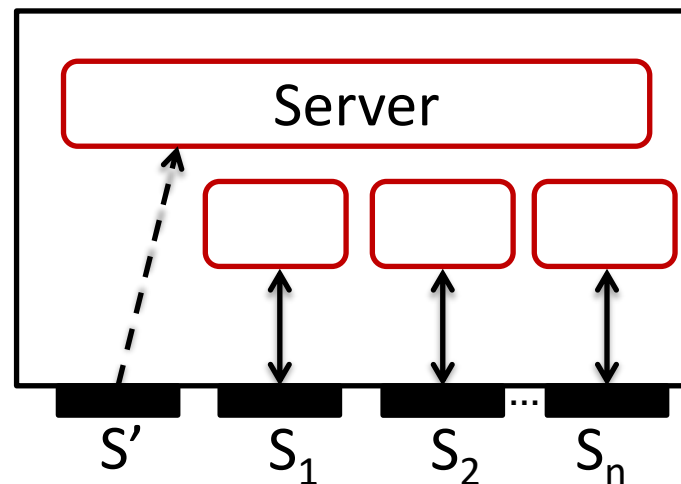
■ Vantaggi:

- Efficienza nell'uso delle risorse (consumo di CPU e RAM basso)
- Costi di context switching abbattuti (tempo sprecato nel passaggio da un processo o thread a un altro)
- Flusso del programma deterministico (facile eseguire il debugging)

■ Svantaggi:

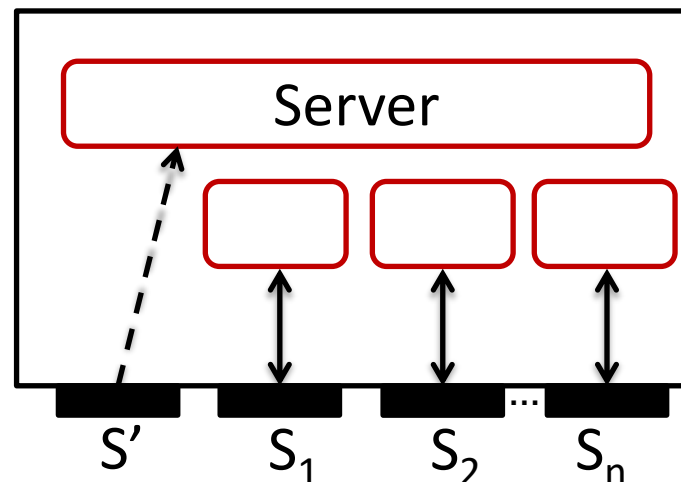
- Nel caso di computazione pesante per un client gli altri client rimangono bloccati
- No supporto multi-core
- Complessità della logica per evitare un comportamento "bloccante" (responsabilità del programmatore)

Tipi di server - Server concorrente multi-processo



- Il server in ascolto sulla porta di ascolto crea un nuovo processo figlio nel momento in cui è accettata una connessione con **accept()**
 - Viene eseguita una **fork()**, che crea una copia esatta del server
- Il figlio gestisce il client specifico, quindi termina
- Il padre torna immediatamente in ascolto in attesa di una nuova richiesta di connessione

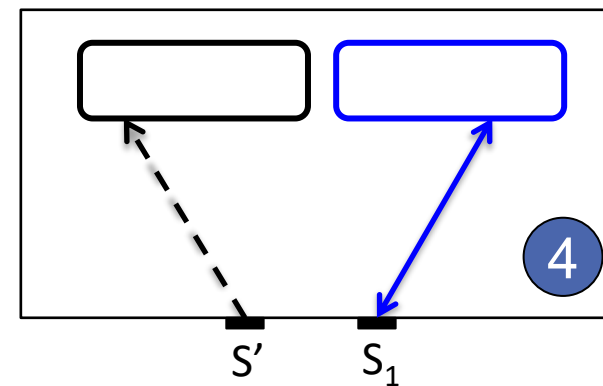
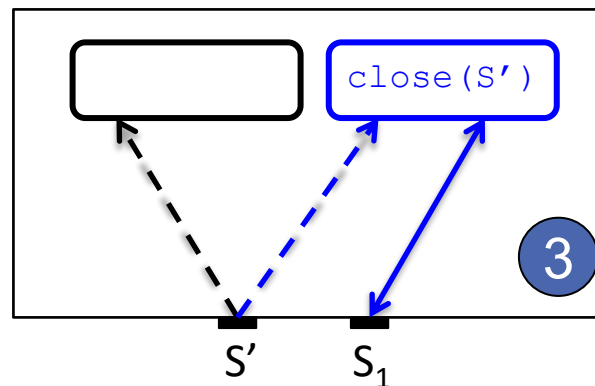
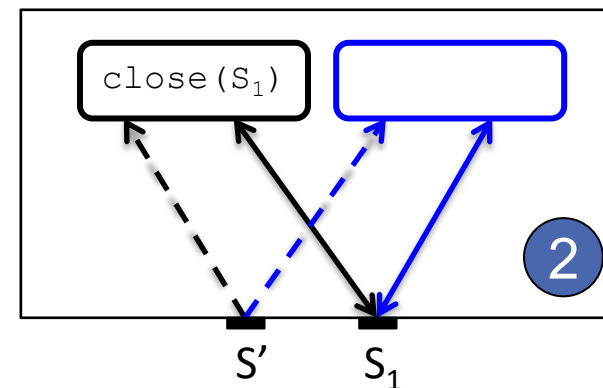
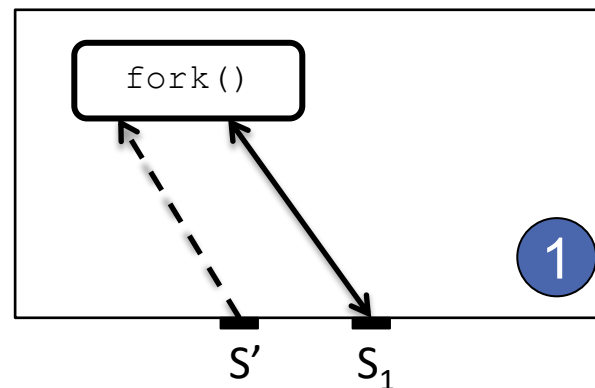
Tipi di server - Server concorrente multi-processo



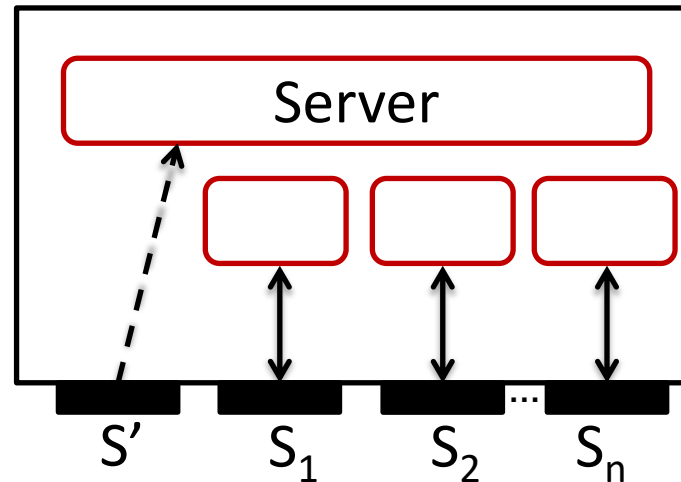
- **Problema:** La `fork()` crea un processo clone del padre che ne eredita i canali di I/O, incluse le socket
- Il codice deve prevedere quindi che:
 - Il padre chiuda la socket di servizio usata per la comunicazione con il client
 - Il figlio chiuda la socket di ascolto su cui si rimette in ascolto il padre
- Cosa può succedere altrimenti?
 - La lettura/scrittura su una socket da parte di più processi determina un **problema di concorrenza**: accesso ad una risorsa condivisa (i buffer della socket)

Tipi di server - Server concorrente multi-processo

1. Viene effettuata la **fork()**
2. Il processo padre chiude con **close()** la socket di servizio S_1
3. Il processo figlio chiude con **close()** la socket di ascolto S'
4. I due processi sono pronti:
 - Il padre torna alla fase di ascolto
 - Il figlio inizia la fase di conversazione con il client



Tipi di server - Server concorrente multi-processo



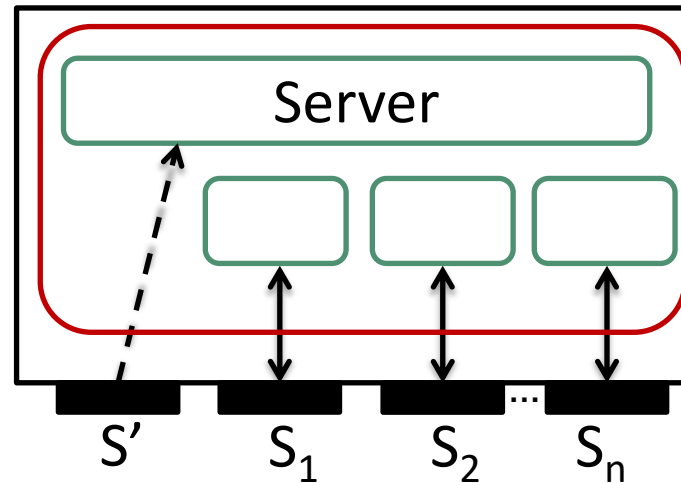
■ Vantaggi:

- Massima robustezza e isolamento totale (se il figlio ha dei problemi, il padre e gli altri figli continuano a funzionare)
- Semplicità di programmazione (la struttura del server è la stessa della versione iterativa)
- Sfruttamento multi-core (processi diversi possono essere eseguiti da CPU diverse)

■ Svantaggi:

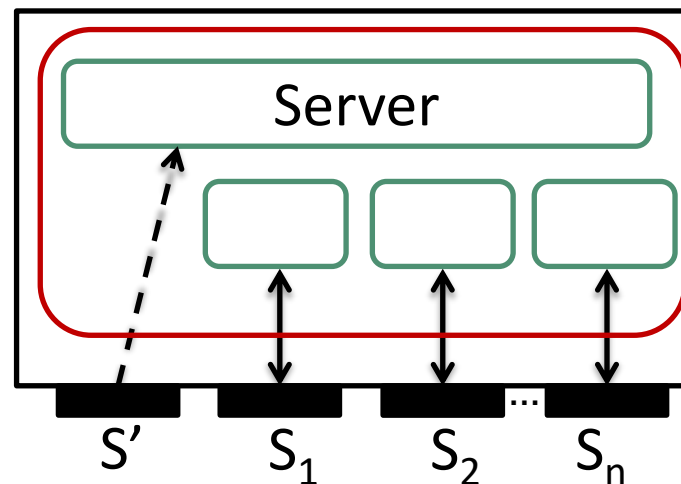
- Consumo elevato di risorse, specialmente RAM
- Lentezza nella creazione di nuovi processi (**fork()** è più lenta di **select()**)
- Costi di context switching (onerosità nel passaggio da un processo all'altro)

Tipi di server - Server concorrente multi-thread



- Il server in ascolto sulla porta di ascolto crea un nuovo thread nel momento in cui è accettata una connessione con **accept()**
 - Lo spazio di memoria è condiviso tra thread
- Il figlio gestisce il client specifico
 - Una volta chiusa la connessione con il client il thread termina
- Il padre torna immediatamente in ascolto in attesa di una nuova richiesta di connessione

Tipi di server - Server concorrente multi-thread



■ Vantaggi:

- Soluzione leggera (creare un thread è più rapido che creare un processo e il consumo di RAM è ridotto)
- Costi di context switching contenuti (più semplice passare da un thread all'altro che da un processo all'altro)
- Sfruttamento multi-core (thread diversi possono essere eseguiti da CPU diverse)

■ Svantaggi:

- Mancanza di isolamento (se un thread ha un errore grave cade l'intero server)
- Rischi di "race conditions", che devono essere gestiti dal programmatore per mezzo di strumenti di sincronizzazione (es. semafori)

Breve confronto tra i modelli

- Quando conviene progettare un server mono-processo (iterativo, concorrente, multi-thread) oppure multi-processo?
 - Ovviamente dipende dalle caratteristiche del servizio
- Mono-processo (iterativo, concorrente, multi-thread)
 - Gli utenti condividono lo stesso spazio di lavoro
 - Adatto ad applicazioni cooperative che prevedono la modifica dello stato (lettura e scrittura)
 - Sistemi di chat, gaming online, editor collaborativi, ...
- Multi-processo
 - Ogni utente ha uno spazio di lavoro autonomo
 - Adatto ad applicazioni cooperative che non modificano lo stato del server (sola lettura)
 - Server web con contenuti statici, ...
 - Adatto ad applicazioni autonome che modificano uno spazio di lavoro proprio (lettura e scrittura)
 - Sistemi di login, gestionali bancari, ...



Server concorrente a processo singolo in Java

EsempioServerConcorrente

Server concorrente: un primo tentativo

- Consideriamo un server che accetta di connettersi con due client, legge dalle rispettive socket di servizio e scrive quanto ricevuto a schermo
- Come può essere strutturato il codice?

Server concorrente: un primo tentativo

```
1. ... // dichiarazioni di variabile
2. Socket[] clientSocket = new Socket[2];
3. DataInputStream[] in = new DataInputStream[2];

4. try {
5.     listenSocket = new ServerSocket(53535);

6.
7.     clientSocket[0] = listenSocket.accept();
8.     clientSocket[1] = listenSocket.accept();

9.     // crea due canali per la lettura
10.    in[0] = new DataInputStream(clientSocket[0].getInputStream());
11.    in[1] = new DataInputStream(clientSocket[1].getInputStream());

12.
13.    int bytesRead = 0;
14.    while (true) {
15.        bytesRead = in[0].read(byteReceived);
16.        if (bytesRead == -1)
17.            break; // non ci sono byte aggiuntivi
18.        messageString += new String(byteReceived, 0, bytesRead);
19.        System.out.println("Received: " + messageString);
20.    }
21.    ... // resto del codice (es. un loop analogo per leggere da in[1] )
```


Server concorrente: un primo tentativo

```
1. ... // dichiarazioni di variabile
2. Socket[] clientSocket = new Socket[2];
3. DataInputStream[] in = new DataInputStream[2];

4. try {
5.     listenSocket = new ServerSocket(53535);
6.     // accetta due connessioni. Cosa succede?
7.     clientSocket[0] = listenSocket.accept();
8.     clientSocket[1] = listenSocket.accept();

9.     // crea due canali per la lettura
10.    in[0] = new DataInputStream(clientSocket[0].getInputStream());
11.    in[1] = new DataInputStream(clientSocket[1].getInputStream());

12.
13.    int bytesRead = 0;
14.    while (true) {
15.        bytesRead = in[0].read(byteReceived);
16.        if (bytesRead == -1)
17.            break; // non ci sono byte aggiuntivi
18.        messageString += new String(byteReceived, 0, bytesRead);
19.        System.out.println("Received: " + messageString);
20.    }
21.    ... // resto del codice (es. un loop analogo per leggere da in[1] )
```

Server concorrente: un primo tentativo

```
1. ... // dichiarazioni di variabile
2. Socket[] clientSocket = new Socket[2];
3. DataInputStream[] in = new DataInputStream[2];

4. try {
5.     listenSocket = new ServerSocket(53535);
6.     // accetta due connessioni. Cosa succede?
7.     clientSocket[0] = listenSocket.accept();
8.     clientSocket[1] = listenSocket.accept();

9.     // crea due canali per la lettura
10.    in[0] = new DataInputStream(clientSocket[0].getInputStream());
11.    in[1] = new DataInputStream(clientSocket[1].getInputStream());

12.    // cosa succede quando si inizia a leggere da uno dei due canali?
13.    int bytesRead = 0;
14.    while (true) {
15.        bytesRead = in[0].read(byteReceived);
16.        if (bytesRead == -1)
17.            break; // non ci sono byte aggiuntivi
18.        messageString += new String(byteReceived, 0, bytesRead);
19.        System.out.println("Received: " + messageString);
20.    }
21.    ... // resto del codice (es. un loop analogo per leggere da in[1] )
```

Server concorrente: un primo tentativo

```
1. ... // dichiarazioni di variabile
2. Socket[] clientSocket = new Socket[2];
3. DataInputStream[] in = new DataInputStream[2];

4. try {
5.     listenSocket = new ServerSocket(53535);
6.     // accetta due connessioni. Cosa succede?
7.     clientSocket[0] = listenSocket.accept();
8.     clientSocket[1] = listenSocket.accept();

9.     // crea due canali per la lettura
10.    in[0] = new DataInputStream(clientSocket[0].getInputStream());
11.    in[1] = new DataInputStream(clientSocket[1].getInputStream());

12.    // cosa succede quando si inizia a leggere da uno dei due canali?
13.    int bytesRead = 0;
14.    while (true) {
15.        bytesRead = in[0].read(byteReceived);
16.        if (bytesRead == -1)
17.            break; // non ci sono byte aggiuntivi
18.        messageString += new String(byteReceived, 0, bytesRead);
19.        System.out.println("Received: " + messageString);
20.    }
21.    ... // resto del codice (es. un loop analogo per leggere da in[1] )
```

Questo server **non** è concorrente.
Come possiamo cambiare il codice
per renderlo tale?

I/O non bloccante in Java

- Possibilità introdotta dal package New Input Output (NIO), rilasciato con Java 1.4
- Viene utilizzata l'astrazione dei **canali** (`Channel`), che possono operare in modo bloccante o non bloccante
 - Un canale non bloccante non mette il chiamante in attesa
 - L'operazione richiesta o viene completata immediatamente o restituisce come risultato che nulla è stato fatto
- Solo canali di I/O di tipo socket possono essere usati nei due modi
- I canali socket possono essere usati in modo non bloccante grazie ai `Selector`
 - I `Selector` gestiscono più canali in multiplex

I/O non bloccante in Java

- Package fondamentale:

- `java.nio.channels`

- Classi fondamentali:

- `java.nio.channels.ServerSocketChannel;`
- `java.nio.channels.SocketChannel;`
- `java.nio.channels.DatagramChannel;`
- `java.nio.channels.SelectionKey;`
- `java.nio.channels.Selector;`

Riferimenti principali:

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/nio/channels/package-summary.html>

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/nio/channels/Selector.html>

<https://developer.ibm.com/tutorials/j-nio/>

I selettori (Java selector)

- Un selettore (`Selector`) permette di gestire dei canali selezionati in Java (`SelectableChannel`) con una **`select()`**
- Un selettore può essere creato invocando il metodo `open()` della classe `Selector`:

```
Selector selector = Selector.open();
```

- I canali da monitorare con la **`select()`** devono essere
 1. Messi in modalità non bloccante
 2. Registrati con il metodo `register()`

- Esempio:

```
channel.configureBlocking(false);  
SelectionKey key = channel.register(selector, SelectionKey.OP_READ);
```

- Gli oggetti della classe `SelectionKey` (chiavi) includono:
 - L'informazione relativa al canale
 - L'informazione relativa al selettore
 - L'insieme delle operazioni da monitorare per il canale (prossima slide)
 - L'insieme delle operazioni che sono effettivamente accadute (Ready Set)

I selettori (Java selector)

- Esempio (continua):

```
channel.configureBlocking(false);  
SelectionKey key = channel.register(selector, SelectionKey.OP_READ);
```

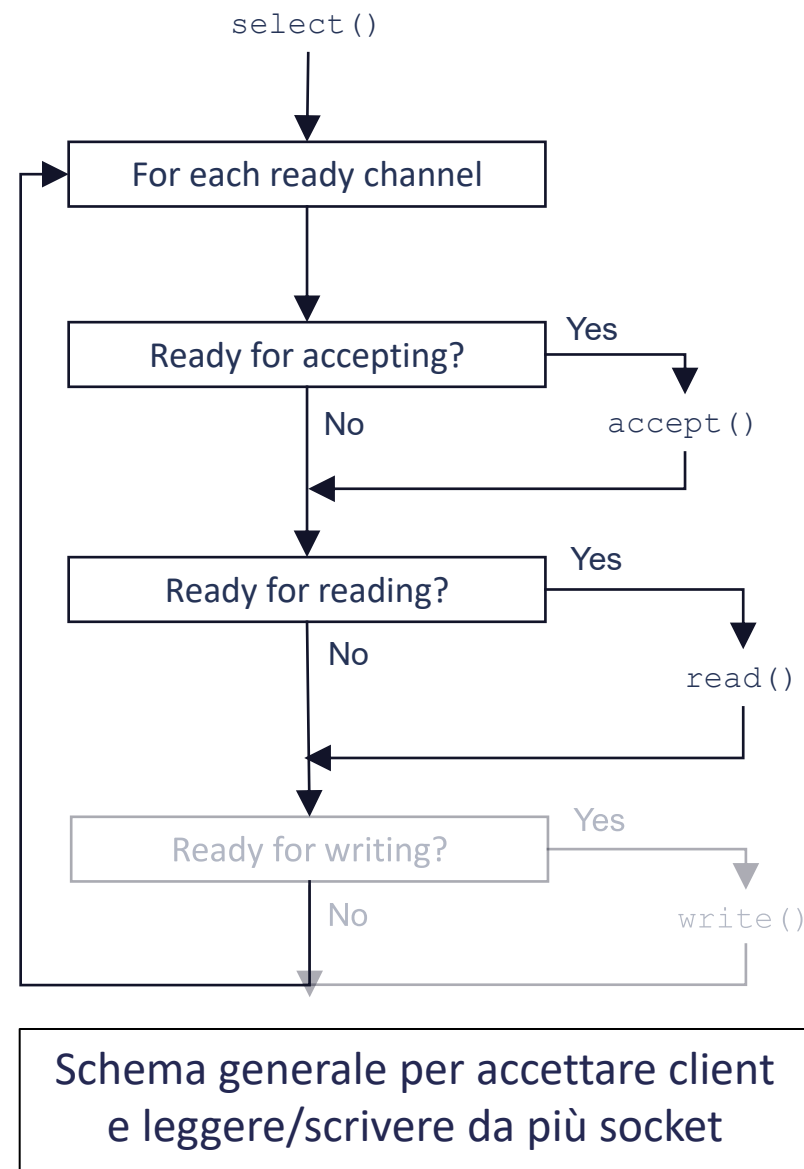
- Ogni canale può essere registrato per monitorare quattro tipo di eventi, definiti da costanti nella classe `SelectionKey`
 - `SelectionKey.OP_CONNECT`: il canale è pronto per completare la sua procedura di connessione al server (connect)
 - `SelectionKey.OP_ACCEPT`: il canale è pronto ad accettare una nuova connessione in entrata da parte di un client (accept)
 - `SelectionKey.OP_READ`: il canale ha dati pronti per essere letti nel suo buffer di ricezione (read)
 - `SelectionKey.OP_WRITE`: il canale è pronto per l'operazione di scrittura (write)

Struttura di un server concorrente - Pseudocodice

```

1.  create ServerSocketChannel;
2.  create Selector;
3.  set the ServerSocketChannel in non-blocking mode
4.  associate the ServerSocketChannel with the Selector;
5.  while(true) {
6.      waiting events from the Selector;
7.      events arrived;
8.      update and retrieve keys related to channels' events;
9.      for each key retrieved by the Selector {
10.         check the type of request;
11.         isAcceptable:
12.             get the client SocketChannel;
13.             associate that SocketChannel with the Selector;
14.             record it for read/write operations;
15.             continue;
16.         isReadable:
17.             get the client SocketChannel;
18.             read from the socket;
19.             process the read data;
20.             continue;
21.         isWriteable:
22.             get the client SocketChannel;
23.             write on the socket;
24.             continue;
25.     }
26. }

```



SenderClient (1)

```
1. import java.io.IOException;
2. import java.io.PrintWriter;
3. import java.net.InetAddress;
4. import java.net.Socket;
5. import java.util.Scanner; // classe per leggere e parsare dati testuali in ingresso in tipi primitivi

6. public class SenderClient {
7.     private Socket socket;
8.     private Scanner scanner;

9.     private SenderClient(InetAddress serverAddress, int serverPort) throws Exception {
10.         ... // vedere prossima slide
12.     }

13.     private void start() throws IOException, InterruptedException {
14.         ... // vedere prossima slide
25.     }

26.     public static void main(String[] args) throws Exception { // crea e avvia il client
27.         SenderClient client = new SenderClient(InetAddress.getByName(args[0]), Integer.parseInt(args[1]));
28.         System.out.println("Connected to: " + client.socket.getInetAddress());
29.         client.start();
30.     }
31. }
```

SenderClient (2)

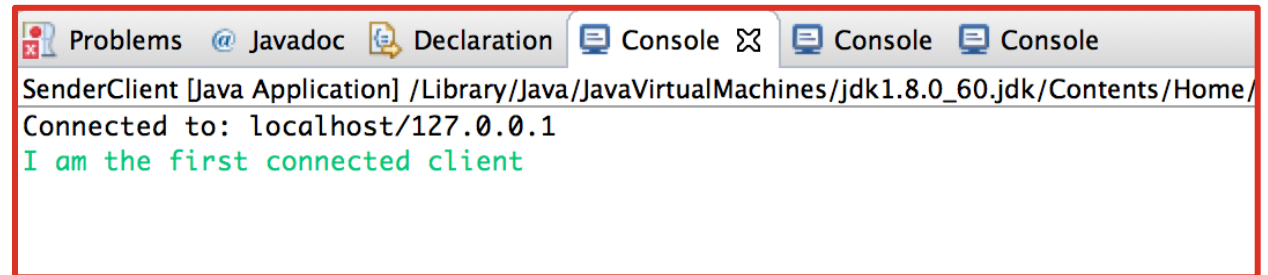
```
9.     private SenderClient(InetAddress serverAddress, int serverPort) throws Exception {
10.         this.socket = new Socket(serverAddress, serverPort);
11.         this.scanner = new Scanner(System.in); //inizializzazione: collego lo scanner alla tastiera
12.     }

13.     private void start() throws IOException, InterruptedException {
14.         String input;
15.         PrintWriter out = new PrintWriter(this.socket.getOutputStream(), true);
16.         while (true) {
17.             input = scanner.nextLine(); //aspetta che l'utente scriva un'intera riga di testo e preme Invio
18.             if (input.contentEquals("exit"))
19.                 break;
20.             out.print(input);
21.             out.flush();
22.         }
23.         System.out.println("Client terminate.");
24.         socket.close();
25.     }
```

SenderClient (2)

```
9.     private SenderClient(InetAddress serverAddress, int serverPort) throws Exception {
10.         this.socket = new Socket(serverAddress, serverPort);
11.         this.scanner = new Scanner(System.in); //inizializzazione: collego lo scanner alla tastiera
12.     }

13.     private void start() throws IOException, InterruptedException {
14.         String input;
15.         PrintWriter out = new PrintWriter(this.socket.getOutputStream(), true);
16.         while (true) {
17.             input = scanner.nextLine(); //aspetta che l'utente scriva un'intera riga di testo e preme Invio
18.             if (input.contentEquals("exit"))
19.                 break;
20.             out.print(input);
21.             out.flush();
22.         }
23.         System.out.println("Client terminate.");
24.         socket.close();
25.     }
```



ConcurrentServer (1)

```
1. public class ConcurrentServer {
2.     public static void main(String[] args) throws IOException {
3.         Selector selector = Selector.open(); // viene creato il selettore
4.         ServerSocketChannel serverSocket = ServerSocketChannel.open(); // viene creato il canale serverSocket
5.         serverSocket.bind(new InetSocketAddress("localhost", 5353)); // bind() su localhost e porta 5353
6.         serverSocket.configureBlocking(false); // serverSocket settata in modalità non bloccante
7.         serverSocket.register(selector, SelectionKey.OP_ACCEPT); // serverSocket registrata al selettore (accept)

8.         while (true) {
9.             selector.select(); // chiamo la select()
10.            Set<SelectionKey> selectedKeys = selector.selectedKeys(); // ogni chiave (key) in selectedKeys si
            riferisce a canali registrati e pronti per un'operazione (chiave con Ready Set non vuoto)
11.            Iterator<SelectionKey> iter = selectedKeys.iterator(); //creo un iteratore per iterare sulle
            selectedKeys (canali pronti)
12.            while (iter.hasNext()) { // itero su tutte le chiavi in selectedKeys
13.                SelectionKey key = iter.next(); // considero la chiave corrente
14.                if (key.isAcceptable()) { // se la chiave è relativa a una richiesta di connessione
15.                    acceptClientRequest(selector, serverSocket); //funzione per chiamare accept() e stabilire la
                    connessione con il client (prossima slide)
16.                }
17.                if (key.isReadable()) { // serve un client connesso (dati ricevuti al buffer)
18.                    readClientBytes(key); // funzione per leggere ed elaborare i dati ricevuti dal client (prossima
                    slide)
19.                }
20.                iter.remove(); // rimuovi la chiave corrente dall'insieme
21.            } } }
```

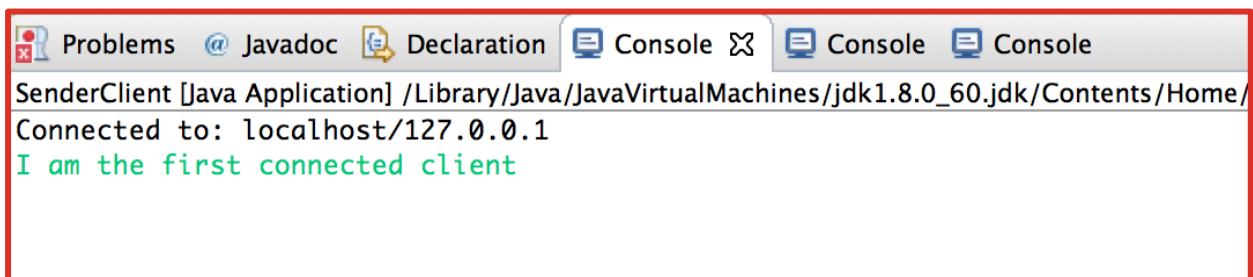
ConcurrentServer (2)

```
22. private static void acceptClientRequest(Selector selector, ServerSocketChannel serverSocket)
23.                                     throws IOException {
24.     SocketChannel client = serverSocket.accept(); //crea il canale client (socket di servizio)
25.     client.configureBlocking(false); // socket di servizio "client" settata in modalità non bloccante
26.     client.register(selector, SelectionKey.OP_READ); // registra al selettore per operazioni di lettura
31. }

32. private static void readClientBytes(SelectionKey key) throws IOException {
33.     SocketChannel client = (SocketChannel) key.channel(); //recupera la socket associata a key ("client")
34.     int BUFFER_SIZE = 256; // definisci la dimensione del buffer in ricezione dell'applicazione in byte
35.     ByteBuffer buffer = ByteBuffer.allocate(BUFFER_SIZE); // istanzia il buffer
36.     try { // leggi i byte in arrivo dal client (semplificato, servirebbe un ciclo di lettura!)
37.         if (client.read(buffer) == -1) { // se la connessione è stata chiusa dal client
38.             client.close(); // chiudi la socket di servizio
39.             return;
40.         }
41.     } catch (Exception e) { // gestione dell'eccezione se il client non è più attivo
42.         e.printStackTrace();
43.         return;
44.     }
45.     // mostra i byte a schermo usando il numero di porta del client per discriminarli
46.     buffer.flip(); // imposta il limite di byte alla posizione corrente e la posizione corrente a 0
47.     Charset charset = Charset.forName("UTF-8"); // definizione del set di caratteri per la decodifica
48.     CharsetDecoder decoder = charset.newDecoder(); // crea un traduttore di caratteri UTF-8
49.     CharBuffer charBuffer = decoder.decode(buffer); // trasforma i byte in caratteri
50.     System.out.println(client.socket().getPort() + ": " + charBuffer.toString()); // stampa a monitor
51. }
```

Esecuzione concorrente

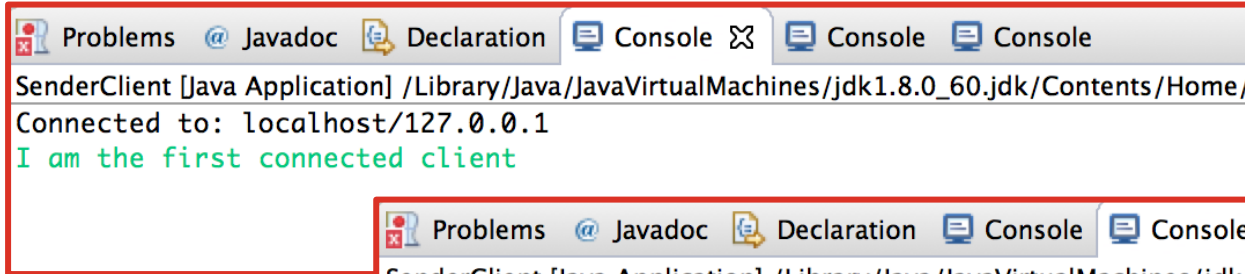
Esecuzione concorrente



The screenshot shows an IDE interface with a console window. The console has tabs for 'Problems', 'Javadoc', 'Declaration', and three 'Console' tabs. The active console tab displays the following text:

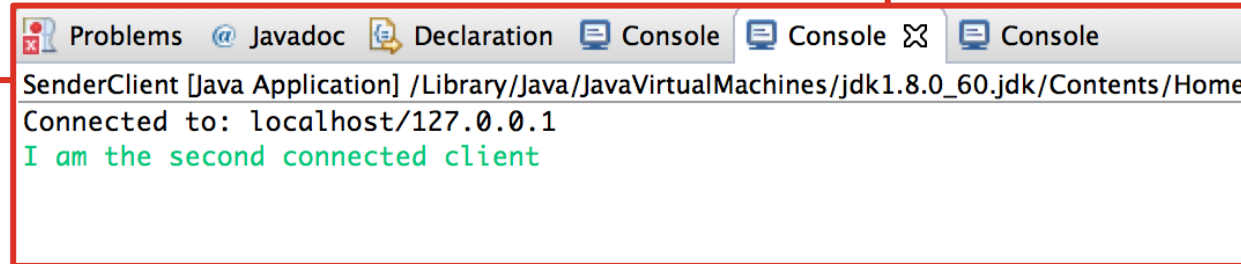
```
SenderClient [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_60.jdk/Contents/Home/  
Connected to: localhost/127.0.0.1  
I am the first connected client
```

Esecuzione concorrente



Problems Javadoc Declaration Console Console Console

SenderClient [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_60.jdk/Contents/Home/
Connected to: localhost/127.0.0.1
I am the first connected client



Problems Javadoc Declaration Console Console Console

SenderClient [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_60.jdk/Contents/Home/
Connected to: localhost/127.0.0.1
I am the second connected client

Esecuzione concorrente

```
Problems @ Javadoc Declaration Console Console Console
SenderClient [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_60.jdk/Contents/Home/
Connected to: localhost/127.0.0.1
I am the first connected client
```

```
Problems @ Javadoc Declaration Console Console Console
SenderClient [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_60.jdk/Contents/Home/
Connected to: localhost/127.0.0.1
I am the second connected client
```

```
Problems @ Javadoc Declaration Console Console Console
ConcurrentServer [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_60.jdk/Contents/Home
63845: I am the first connected client
63846: I am the second connected client
```

Esecuzione concorrente

```
Problems @ Javadoc Declaration Console Console Console
SenderClient [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_60.jdk/Contents/Home/
Connected to: localhost/127.0.0.1
I am the first connected client
```

```
Problems @ Javadoc Declaration Console Console Console
SenderClient [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_60.jdk/Contents/Home/
Connected to: localhost/127.0.0.1
I am the second connected client
```

```
Problems @ Javadoc Declaration Console Console Console
ConcurrentServer [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_60.jdk/Contents/Home/
63845: I am the first connected client
63846: I am the second connected client
```

```
Problems @ Javadoc Declaration Console Console Console
<terminated> SenderClient [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_60.jdk/Contents/Home/
Connected to: localhost/127.0.0.1
I am the first connected client
exit
Client terminate.
```

Esecuzione concorrente

```
Problems @ Javadoc Declaration Console Console Console
SenderClient [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_60.jdk/Contents/Home/
Connected to: localhost/127.0.0.1
I am the first connected client
```

```
Problems @ Javadoc Declaration Console Console Console
SenderClient [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_60.jdk/Contents/Home/
Connected to: localhost/127.0.0.1
I am the second connected client
```

```
Problems @ Javadoc Declaration Console Console Console
ConcurrentServer [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_60.jdk/Contents/Home/
63845: I am the first connected client
63846: I am the second connected client
```

```
Problems @ Javadoc Declaration Console Console Console
<terminated> SenderClient [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_60.jdk/Contents/Home/
Connected to: localhost/127.0.0.1
I am the first connected client
exit
Client terminate.
```

```
Problems @ Javadoc Declaration Console Console Console
SenderClient [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_60.jdk/Contents/Home/
Connected to: localhost/127.0.0.1
I am the latest client
```

Esecuzione concorrente

Problems Javadoc Declaration Console Console Console
 SenderClient [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_60.jdk/Contents/Home/
 Connected to: localhost/127.0.0.1
 I am the first connected client

Problems Javadoc Declaration Console Console Console
 SenderClient [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_60.jdk/Contents/Home/
 Connected to: localhost/127.0.0.1
 I am the second connected client

Problems Javadoc Declaration Console Console Console
 ConcurrentServer [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_60.jdk/Contents/Home/
 63845: I am the first connected client
 63846: I am the second connected client

Problems Javadoc Declaration Console Console Console
 <terminated> SenderClient [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_60.jdk/Contents/Home/
 Connected to: localhost/127.0.0.1
 I am the first connected client
 exit
 Client terminate.

Problems Javadoc Declaration Console Console Console
 SenderClient [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_60.jdk/Contents/Home/
 Connected to: localhost/127.0.0.1
 I am the latest client

Problems Javadoc Declaration Console Console Console
 ConcurrentServer [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_60.jdk/Contents/Home/
 63845: I am the first connected client
 63846: I am the second connected client
 63872: I am the latest client



Server concorrenti multi-processo e multi-thread in Java: cenni

Server concorrenti multi-processo

- Come creare un clone di un processo in Java?
- Java non supporta direttamente la **fork()**, ma i processi possono essere clonati sfruttando la classe `java.lang.ProcessBuilder`
- `ProcessBuilder` è usata in generale per creare processi
 - Ogni istanza di `ProcessBuilder` gestisce un insieme di attributi di processo (es. directory di lavoro, variabili di ambiente, ...)
 - Il metodo `start()` crea una nuova istanza di `Process` con tali attributi
 - `start()` può essere invocato in maniera ripetuta dalla stessa istanza di `ProcessBuilder` per creare nuovi sottoprocessi con attributi identici o correlati

Server concorrenti multi-processo

■ Un semplice esempio:

```
//Configura il ProcessBuilder per lanciare program.exe
1. ProcessBuilder pb = new ProcessBuilder("C:\\Windows\\system32\\program.exe");
2. pb.inheritIO(); //Reindirizza l'output del figlio sulla console del genitore per vederlo

3. try {
4.     Process p = pb.start(); // Equivale a fork() e exec() in Unix
5.     p.waitFor(); // Aspetta che il processo figlio completi le sue operazioni
6. } catch (Exception e) {
7.     e.printStackTrace();
8. }
```

■ Questa logica può essere sfruttata per creare un server multi-processo

- Non lo vediamo in dettaglio

Riferimento:

<https://docs.oracle.com/en/java/javase/13/docs/api/java.base/java/lang/ProcessBuilder.html>

Esempio:

<https://stackoverflow.com/questions/30355085/how-do-i-fork-an-external-process-in-java>

Server concorrenti multi-thread

- La struttura logica di un server multi-thread è la seguente:

```
while (true) {  
    accept a connection;  
    create a thread to deal with the client;  
}
```

- E' possibile creare una classe che estende la classe Thread (es. MultiServerThread) e realizza un server dedicato a un client

- Esempio:

```
1. public static class MultiServerThread extends Thread {  
    ...}  
  
...  
2. while (true)  
3.     new MultiServerThread(serverSocket.accept()).start(); // Rimane in  
    attesa di un client (accept); una volta connesso, crea una nuova  
    istanza di MultiServerThread e avvia l'esecuzione parallela
```

- Vedrete qualcosa in più durante il laboratorio

Riferimenti bibliografici

- J. F. Kurose, K. W. Ross, Reti di Calcolatori e Internet – Un approccio top-down, 8° Edizione
 - Sezione 2.7 («Programmazione delle socket: Come creare un'applicazione di rete»)
- M. Van Steen, A. S. Tanenbaum, Distributed Systems, 4th Edition
 - Sezione 4.3.1 («Simple transient messaging with sockets»)
 - Sezione 3.4.1 («Concurrent versus iterative servers»)
- G. Coulouris, J. Dollimore, T. Kindberg, G. Blair, Distributed Systems – Concepts and Design, 5th Edition
 - Sezione 4.2.2 («Sockets»)
- Risorsa aggiuntiva su programmazione di rete (anche concorrente) in Java:
 - E. R. Harold, Java Network Programming – Developing Networked Applications, 4th Edition